Host guide: Smart-RUDP review

For the presenter only. Each page shows one slide of the real deck, what to say, and the question it is likely to bring.

Computer Networks | Faculty: Anuradha G | 9 Oct 2026

Plan for the session

Total talking time about 15 minutes, then questions.

#	Slide	Time	#	Slide	Time
1	Smart-RUDP	0:20	23	Congestion control: AIMD	0:50
2	The problem	0:40	24	Smart-RUDP: a Q-learning window controller	0:50
3	Why we built it	0:50	25	The AI agent in our code	1:00
4	What we built	0:40	26	Neural agent in the C transport	1:00
5	Results at a glance	1:00	27	Literature behind the design	0:30
6	Up to 83% less time than tuned AIMD	0:40	28	How it was tested	0:40
7	Real clock: 15% to 52% faster than tuned AIMD	1:00	29	A fast, faithful C simulator, and what it showed	1:00
8	Seeing the difference: the window	0:40	30	Issues we found along the way	0:50
9	Real Linux kernel: 65% to 83% faster	1:00	31	What we changed to make it better	1:00
10	Changing paths: fastest in 4 of 4 cases	1:00	32	Benchmark results	0:40
11	Fairer than AIMD, and does not starve TCP	1:00	33	Problems we hit and fixed	0:40
12	Works between two real machines	0:45	34	Our first learned agent: an honest result	0:50
13	Network design: OSPF topology in Packet Tracer	0:50	35	More evidence: baselines, SACK, reordering	1:00
14	Packet Tracer results: routes, failover, ACL	0:40	36	Real sockets found what the emulator could not	0:50
15	Wireshark: RUDP dissector and traces	0:40	37	Safety net: hand over to AIMD	0:50

Rules: say the limits before she asks; answer in one or two sentences; if you do not know, say what you would check.

1. Smart-RUDP

Time on this slide: 0:20



IF ASKED Say the three names and the GitHub link once, then move on.

SAY

- Good morning ma'am. We are Harshavardhan, Rohit and Shloak.
- Our project is Smart-RUDP: a reliable transport protocol we built in C on top of UDP, plus a controller that learns how fast to send.

2. The problem

Time on this slide: 0:40



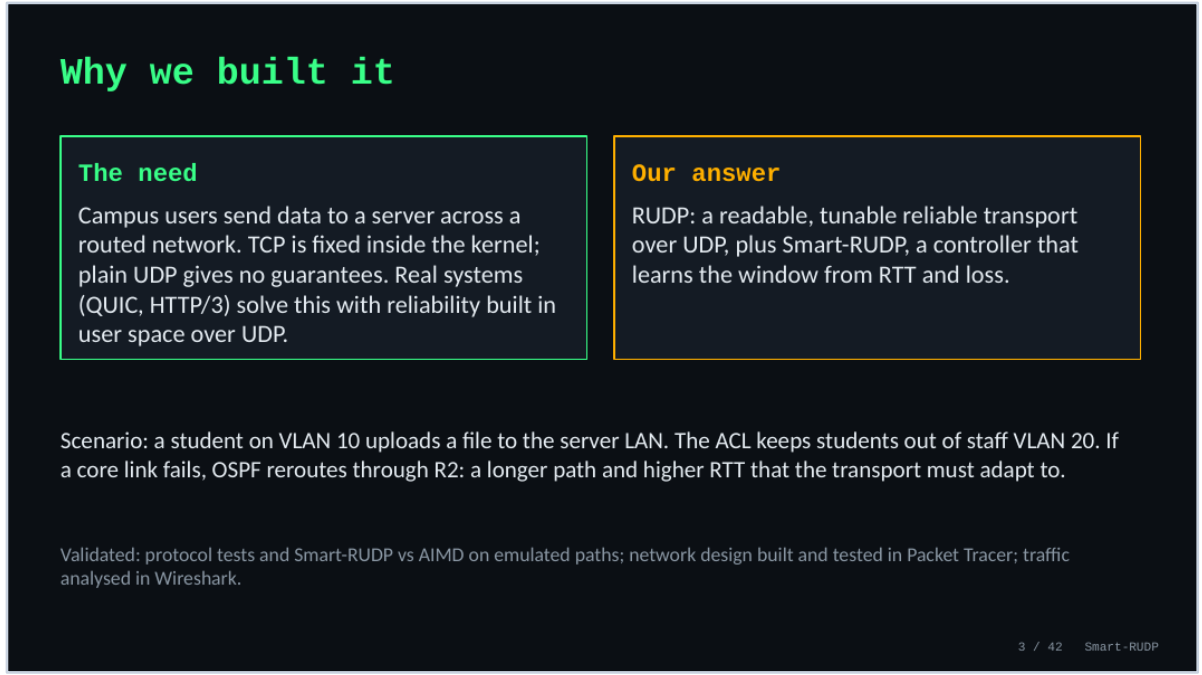
IF ASKED Q: Why not just use TCP? A: TCP cannot be changed or inspected; building our own makes every mechanism visible. QUIC and HTTP/3 do the same in practice.

SAY

- UDP is fast but gives no guarantees: packets can be lost, duplicated or reordered.
- TCP is reliable but sits inside the operating system, so you cannot see or change it.
- In 1986 the early Internet had congestion collapse: throughput fell from 32 Kbps to 40 bps. TCP's congestion control was invented to fix that.
- Our question: can we rebuild TCP's core ideas ourselves, on UDP?

3. Why we built it

Time on this slide: 0:50



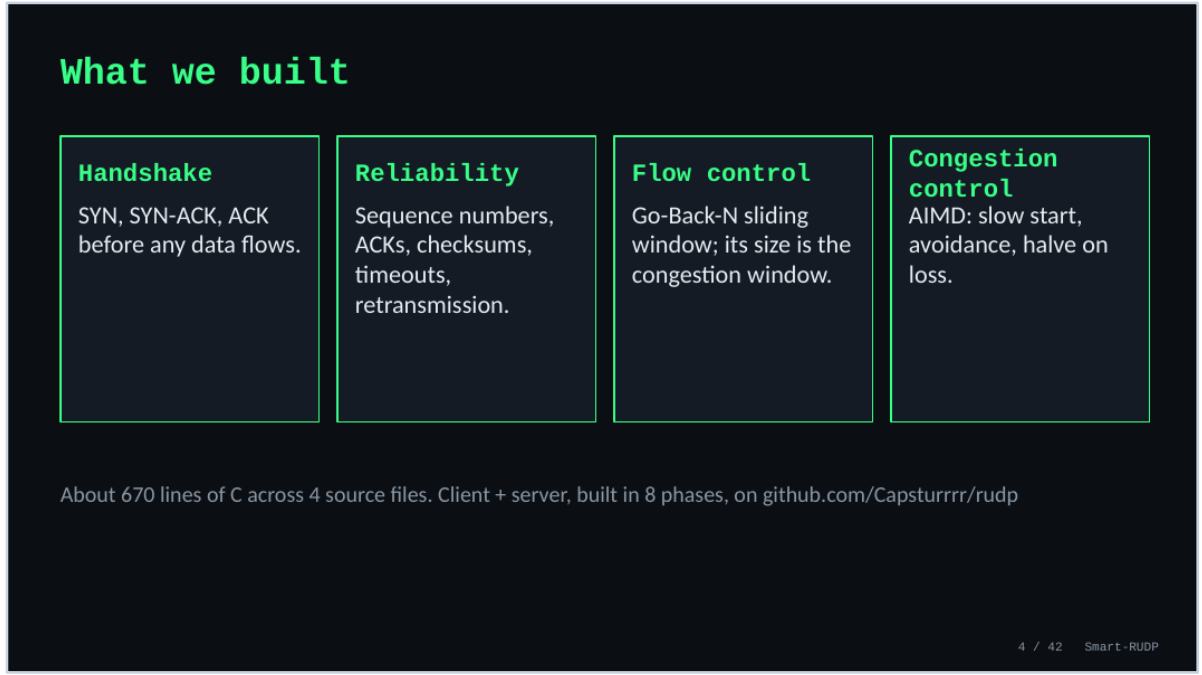
IF ASKED Q: Is this just a lab exercise? A: A lab sends datagrams. We built reliability and congestion control on top, measured them, and used them in an application.

SAY

- The need: campus users send data to a server across a routed network. TCP is fixed in the kernel; UDP gives no guarantees.
- Our answer: RUDP, a readable and tunable transport, and Smart-RUDP, which learns the window from RTT and loss.
- Scenario: a student on VLAN 10 uploads to the server LAN. The ACL keeps students out of staff VLAN 20. If a core link fails, OSPF reroutes through R2: longer path, higher RTT, and the transport must adapt.

4. What we built

Time on this slide: 0:40



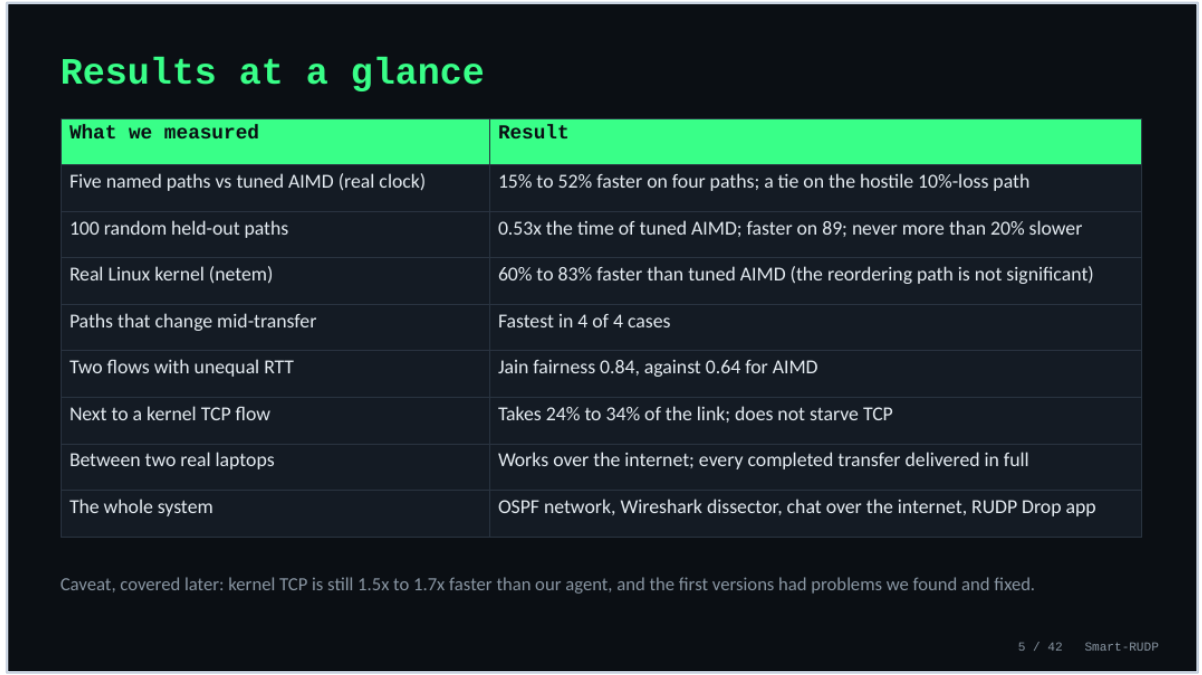
IF ASKED Q: Who did what? A: Agree one answer as a team before you go in.

SAY

- Four pieces: handshake, reliability, flow control and congestion control.
- About 670 lines of C across four source files, built in eight phases, on GitHub at Capsturrrr/rudp.

5. Results at a glance

Time on this slide: 1:00



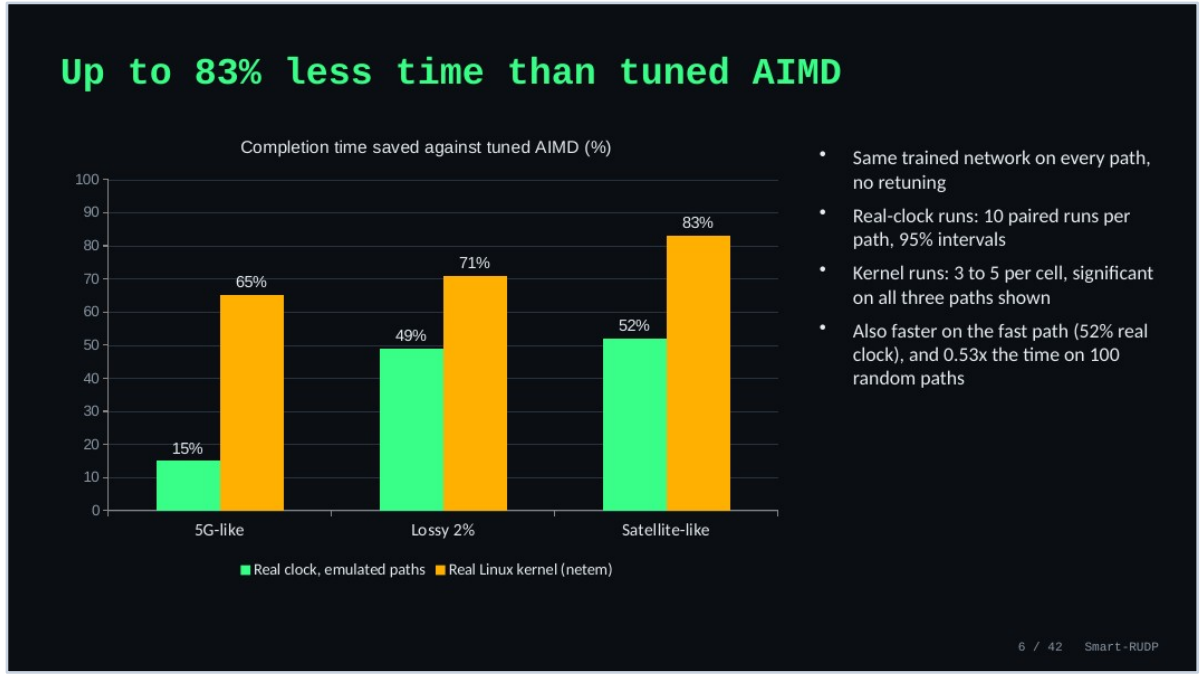
IF ASKED Is kernel TCP faster than your protocol? Yes, 1.5 to 1.7 times on every kernel path. Our claim is that our agent beats tuned AIMD and CUBIC inside our own transport, not that we beat the kernel.

SAY

- Start with the outcome. Against tuned AIMD, the neural agent is 15 to 52 percent faster on four of our five named paths, and ties on the hostile 10 percent loss path. On 100 random paths it needs 0.53 times the time and is never more than 20 percent slower.
- It holds up on the real Linux kernel, 60 to 83 percent faster than tuned AIMD, it copes when the path changes mid-transfer, and it is fairer than AIMD between two flows with different round-trip times. Next to a kernel TCP flow it takes 24 to 34 percent of the link, so it does not starve TCP.
- It also works between two real laptops over the internet, and the whole system is there: the OSPF network, the Wireshark dissector, the chat and the RUDP Drop app. The caveat, which we come back to, is that kernel TCP is still faster.

6. Up to 83% less time than tuned AIMD

Time on this slide: 0:40



IF ASKED Why are kernel gains bigger? The kernel path has real queueing and loss and tuned AIMD is slow to recover there; we report both and do not average them.

SAY

- This is the main result in one picture. Against tuned AIMD, which is a fair, tuned opponent, the neural agent saves 15 percent of the time on the 5G-like path, 49 percent on the lossy path and 52 percent on the satellite-like path on the real clock.
- On the real Linux kernel the savings are bigger, 65, 71 and 83 percent, and significant on all three. It is the same trained network everywhere, with no retuning.

7. Real clock: 15% to 52% faster than tuned AIMD

Time on this slide: 1:00



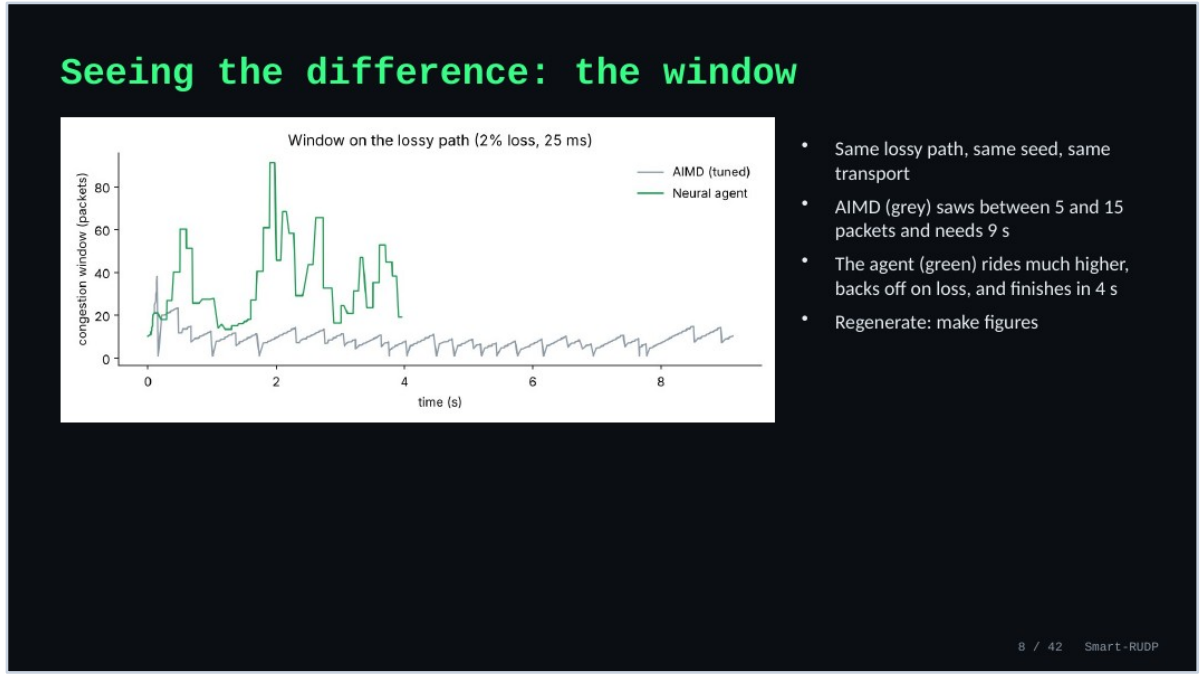
IF ASKED Why did the margin shrink? A retransmission timer defect was hurting AIMD more than the agent. We found it by tracing the retransmissions, fixed it for every policy, and re-ran everything. Without the fix AIMD is 41 percent slower on random paths.

SAY

- This is the final comparison. Everything runs in the C transport with Selective Repeat, a real clock and a real server process. We compare four policies: the original AIMD with a window cap of 32, AIMD tuned with a cap of 128, tuned CUBIC, and the neural agent with its final settings.
- The neural agent is fastest on four of the five paths and significantly faster than tuned AIMD on all four: 15 percent on the 5G-like path, 49 percent on lossy, and 52 percent on the satellite-like and fast paths. It beats CUBIC significantly on lossy and satellite. On the hostile 10 percent loss path nothing is significantly different.
- One correction we want to be upfront about: earlier we reported bigger margins and 262 retransmissions on the fast path. That came from a timer that fired too early. Fixing it helped AIMD even more than the agent, so these numbers are the honest ones.

8. Seeing the difference: the window

Time on this slide: 0:40



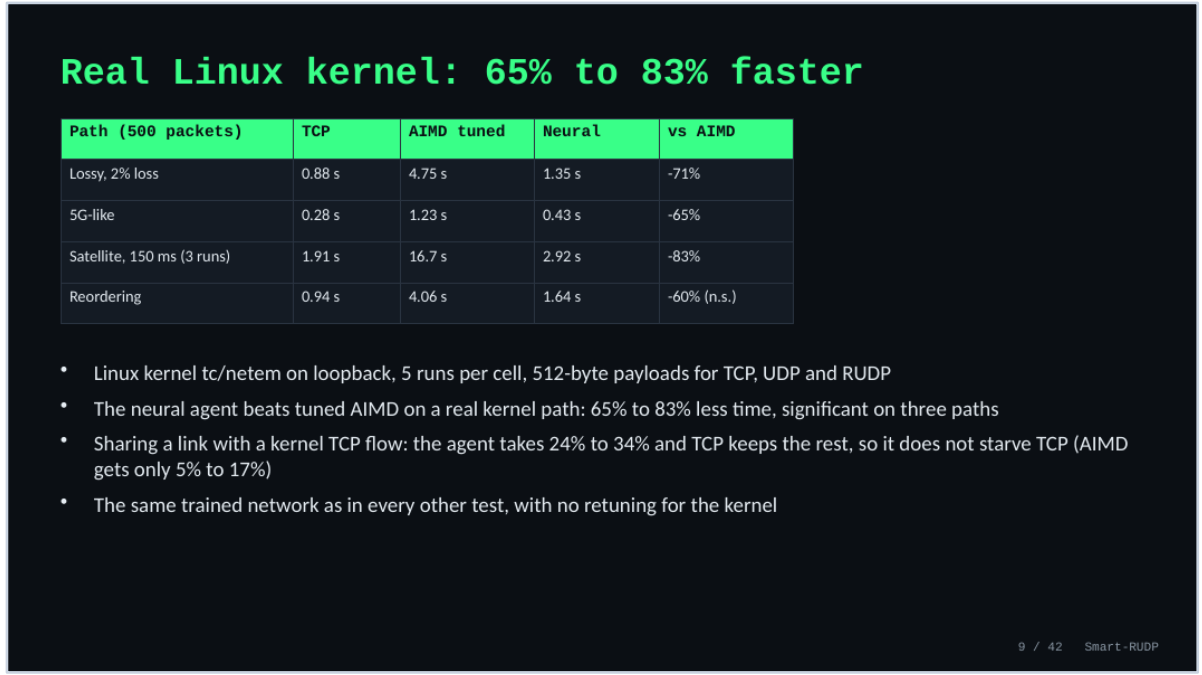
IF ASKED Why does the agent's window jump around so much? It picks one of five multipliers about once per round trip, so it moves in steps; that is the cost of a learned policy with a coarse action set.

SAY

- This is the congestion window on the lossy path, same seed for both. AIMD, in grey, climbs one packet per round trip and halves on every loss, so it never gets above about 15 packets and takes nine seconds. The neural agent, in green, runs much higher, backs off when it sees loss or queueing delay, and finishes in four seconds. The figure comes from our virtual-time sender and can be regenerated with make figures.

9. Real Linux kernel: 65% to 83% faster

Time on this slide: 1:00



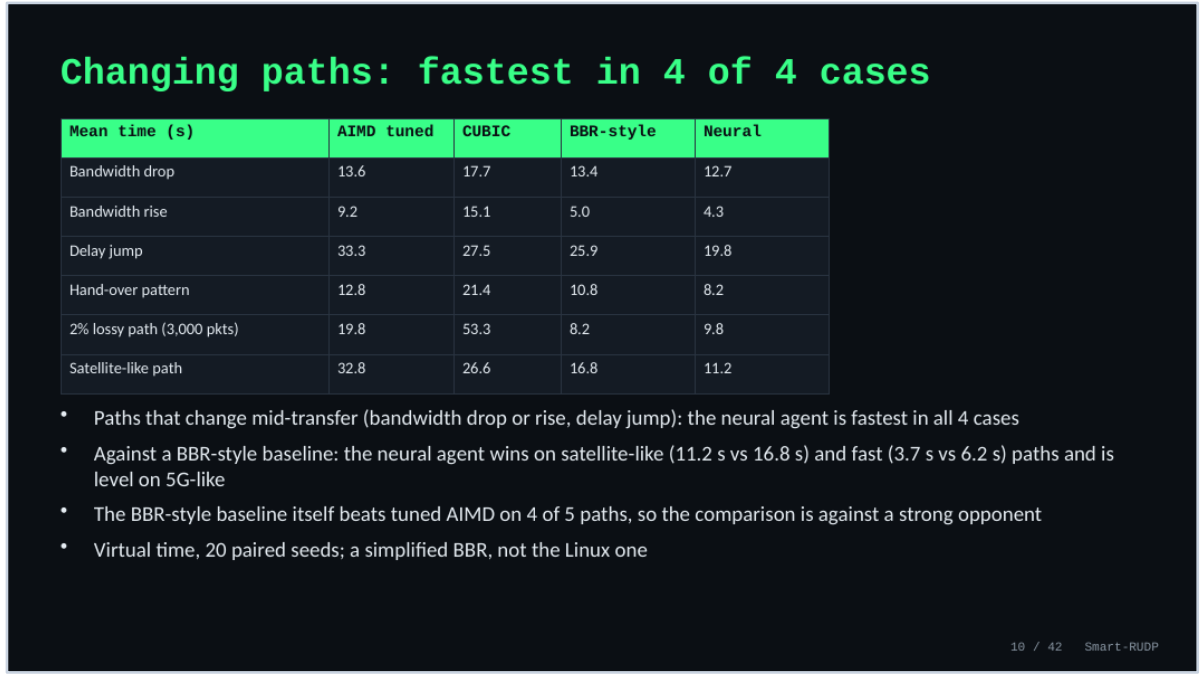
IF ASKED Q: Does it beat TCP? A: No. It beats our AIMD baselines on a real kernel path; mature kernel TCP is faster, by 1.5x to 1.7x. Q: Does it starve TCP? A: No, it takes 24 to 34 percent of a shared link (3 runs per cell, one laptop). Q: Why were the first runs wrong? A: four separate mistakes, the last being 40-byte synthetic packets against 512-byte TCP payloads; all fixed and re-run.

SAY

- Until now our real-socket tests ran through our own proxy. We repeated them on the Linux kernel itself, using tc netem, with kernel TCP in the same table.
- Against tuned AIMD the neural agent is 71 percent faster on the lossy path, 65 percent on the 5G-like path and 83 percent on the satellite path, all significant. On the reordering path it is 60 percent faster but that is not significant.
- Kernel TCP is still faster than our agent on every path, by one and a half to one and seven tenths. We are not claiming to beat TCP.
- We also put a kernel TCP flow and one RUDP flow on the same link. The neural agent takes 24 to 34 percent of it, so it does not starve TCP; tuned AIMD gets only 5 to 17 percent.
- Our first runs were wrong four times: the netem queue limit counted packets in flight, the baseline was untuned, the payload sizes differed, and finally RUDP was sending 40-byte synthetic packets. We found each one, fixed it, re-ran everything and documented it.

10. Changing paths: fastest in 4 of 4 cases

Time on this slide: 1:00



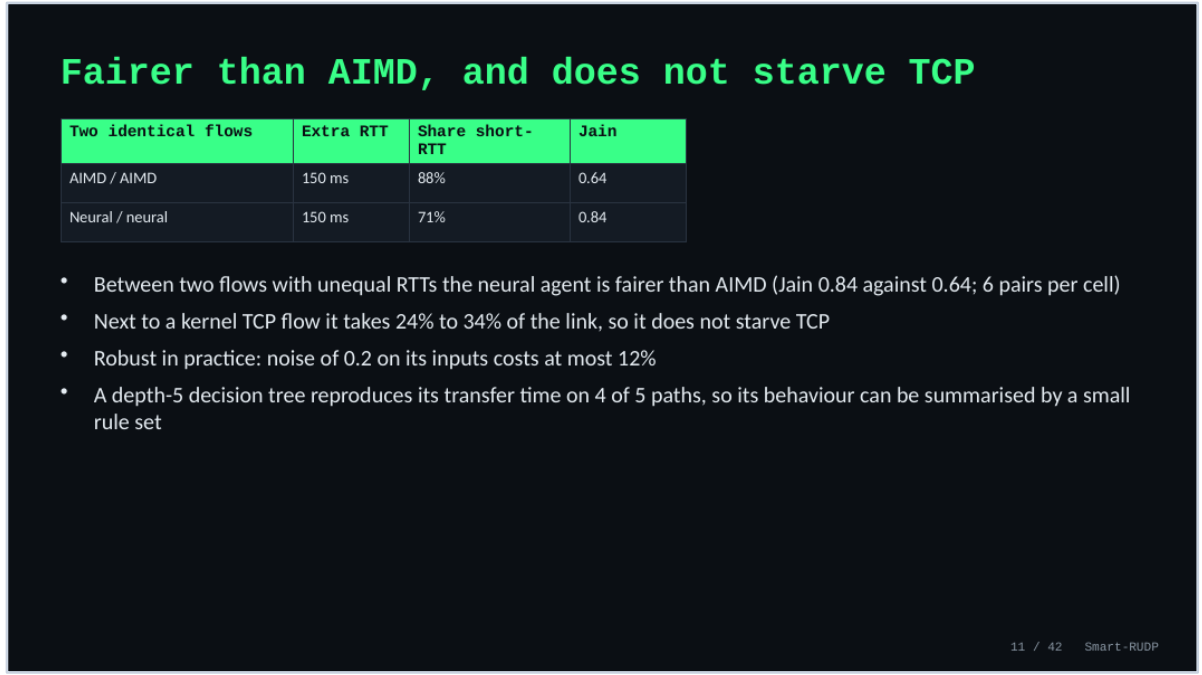
IF ASKED Is your BBR the real one? No: a simplified version with no ProbeRTT and no BBRv3 loss response; the comparison is against a BBR-style policy.

SAY

- Two more research gaps we tested.
- First, paths that change during a transfer: we added a --dyn option so bandwidth and delay change mid-run.
- The neural agent was fastest in all four cases, 6 to 53 percent faster than tuned AIMD, but it retransmits 1 to 6 percent of packets.
- Second, a stronger baseline: a simplified BBR.
- It beats tuned AIMD on four of five paths and is actually faster than our agent on the 2 percent lossy path, 8.2 seconds against 9.8.
- Be honest that our BBR is a simplification and that two bugs in it were found and fixed by these tests.

11. Fairer than AIMD, and does not starve TCP

Time on this slide: 1:00



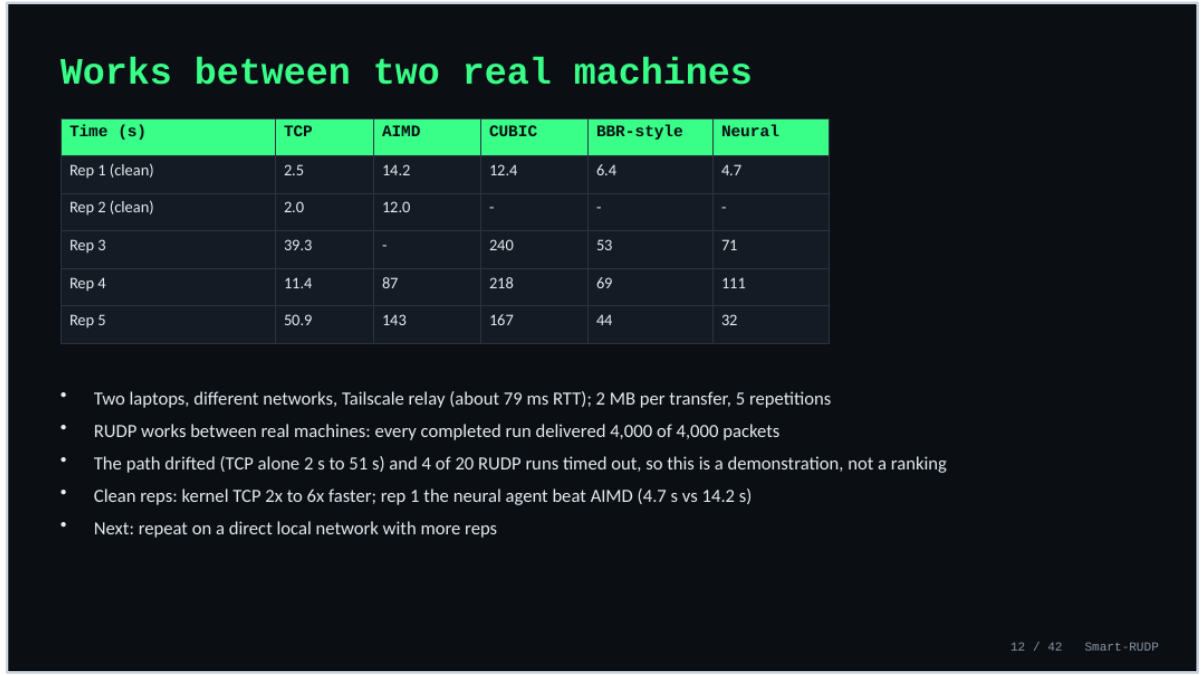
IF ASKED Did you test against kernel TCP? Yes, on kernel netem: the neural agent took 24 to 34 percent of a shared link, three runs per cell on one laptop.

SAY

- Fairness and trust.
- Two neural flows with unequal round-trip times share the link more evenly than two AIMD flows, Jain 0.84 against 0.64 at 150 ms extra RTT, but still favour the short-RTT flow; six pairs per cell so only the largest gap is clear.
- A small decision tree agrees with the network on only 79 percent of decisions but gives the same transfer time on four of five paths.
- Noise on the inputs costs at most 12 percent.
- Formal certification only works at tiny perturbations, so the agent is robust in practice but not proven.
- Kernel TCP coexistence is measured: the neural agent takes 24 to 34 percent of a shared link, so it does not starve TCP, with only three runs per cell.

12. Works between two real machines

Time on this slide: 0:45



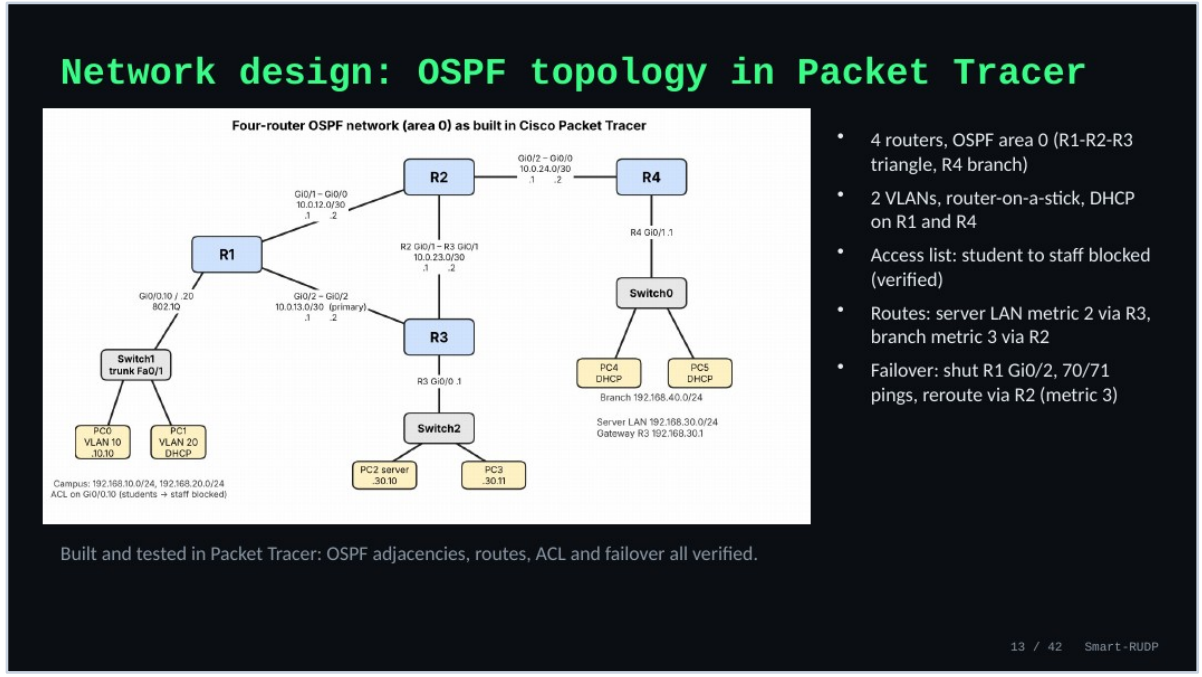
IF ASKED Why not a better test? The other laptop was a one-time setup over the internet; the next step is a direct local network with more repetitions.

SAY

- Until now every test ran on one computer. We ran the sender on one laptop and the receiver on another, on different networks, joined by Tailscale. It could not connect directly, so traffic went through a relay, about 79 milliseconds round trip.
- RUDP works between real machines: every transfer that finished delivered all 4,000 packets, with all four policies.
- Be honest about the numbers: the path drifted, kernel TCP alone took anywhere from 2 to 51 seconds, and 4 of 20 RUDP runs timed out, so this is a demonstration and not a ranking. On the two clean repetitions kernel TCP was 2 to 6 times faster, and in repetition 1 the neural agent beat tuned AIMD, 4.7 against 14.2 seconds.

13. Network design: OSPF topology in Packet Tracer

Time on this slide: 0:50



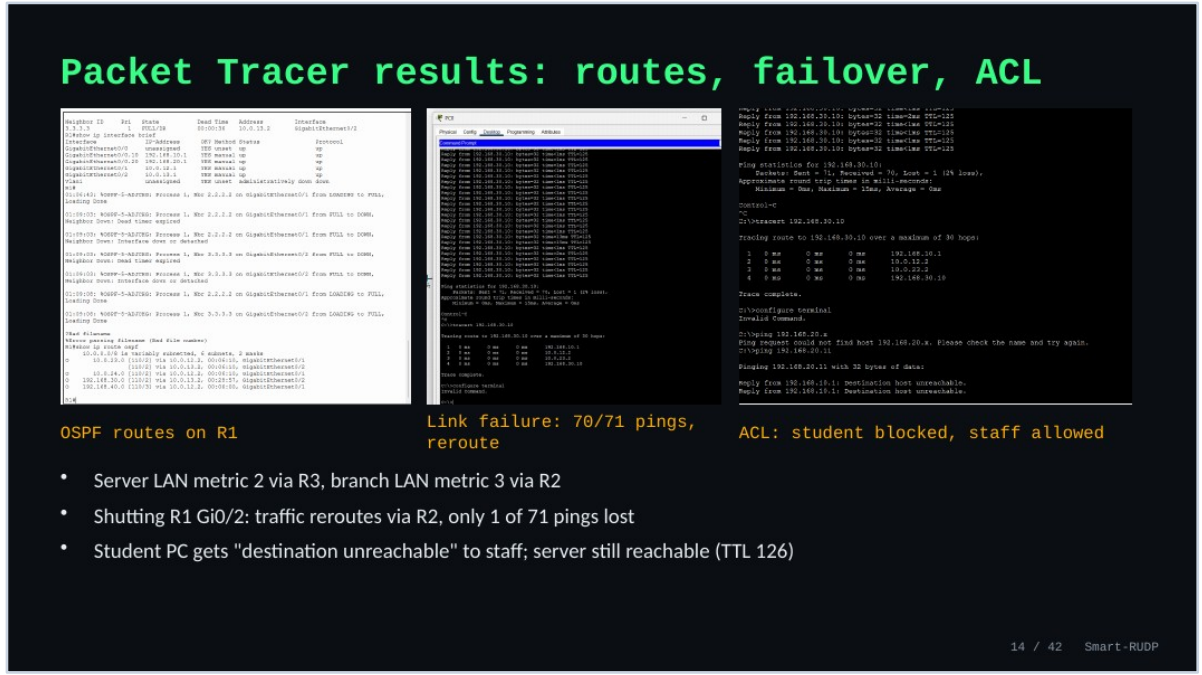
SAY

- Four routers, OSPF area 0: R1, R2, R3 form a triangle and R4 is a branch.
- Two VLANs with router-on-a-stick, DHCP on R1 and R4, and an access list blocking student to staff.
- Server LAN is metric 2 via R3; branch is metric 3 via R2. Failover: shutting R1 Gi0/2 still gave 70 of 71 pings, rerouted via R2.

IF ASKED Q: Does RUDP run inside Packet Tracer? A: No, it cannot run C. Packet Tracer validates the network design; protocol is validated on one machine plus Wireshark.

14. Packet Tracer results: routes, failover, ACL

Time on this slide: 0:40



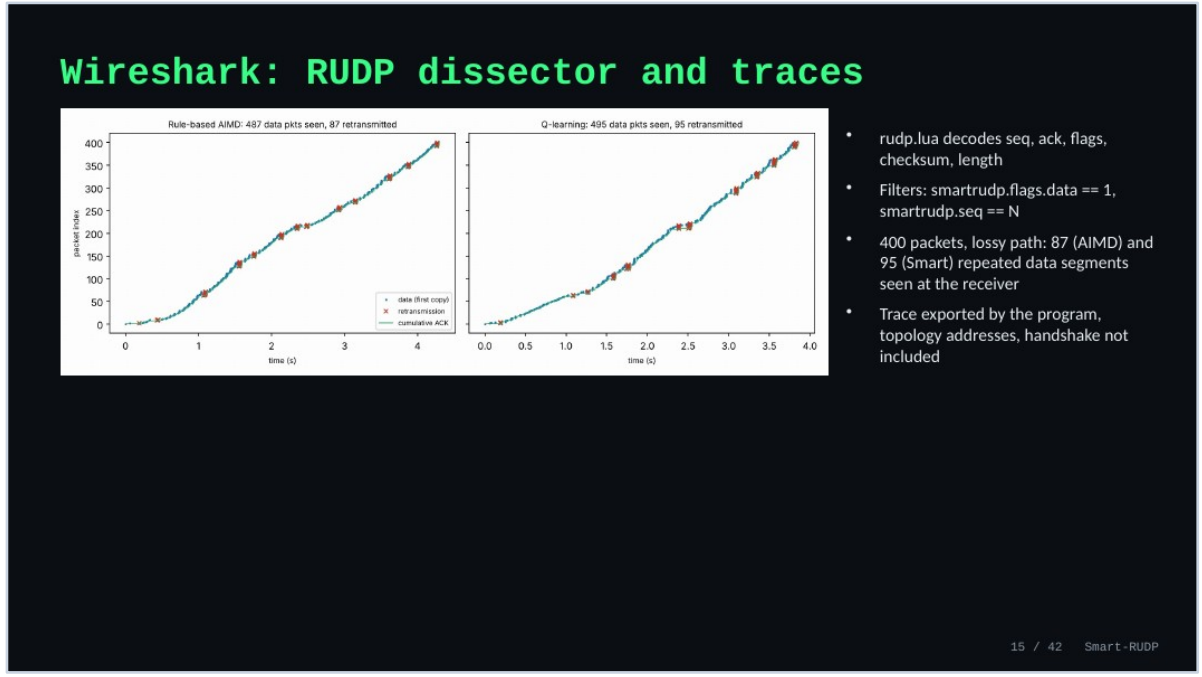
IF ASKED If asked to show it live: have the .pkt file open and follow Demo_Steps.txt.

SAY

- Left: OSPF routes on R1 with the metrics. Middle: the link-failure test, 70 of 71 pings. Right: the ACL, where the student gets destination unreachable to staff but the server is still reachable (TTL 126).

15. Wireshark: RUDP dissector and traces

Time on this slide: 0:40



IF ASKED Q: Why no handshake in the export? A: The export builds synthetic packets. The real tshark capture on the next slide shows the handshake.

SAY

- We wrote a Lua dissector, named smartrudp, that decodes sequence, ack, flags, checksum and length.
- Useful filters: smartrudp.flags.data == 1 and smartrudp.seq == N.
- In a 400-packet lossy run the receiver saw 87 (AIMD) and 95 (Smart) repeated data segments.
- The program's exported trace uses topology addresses and has no handshake.

16. Wireshark: real capture

Time on this slide: 0:50



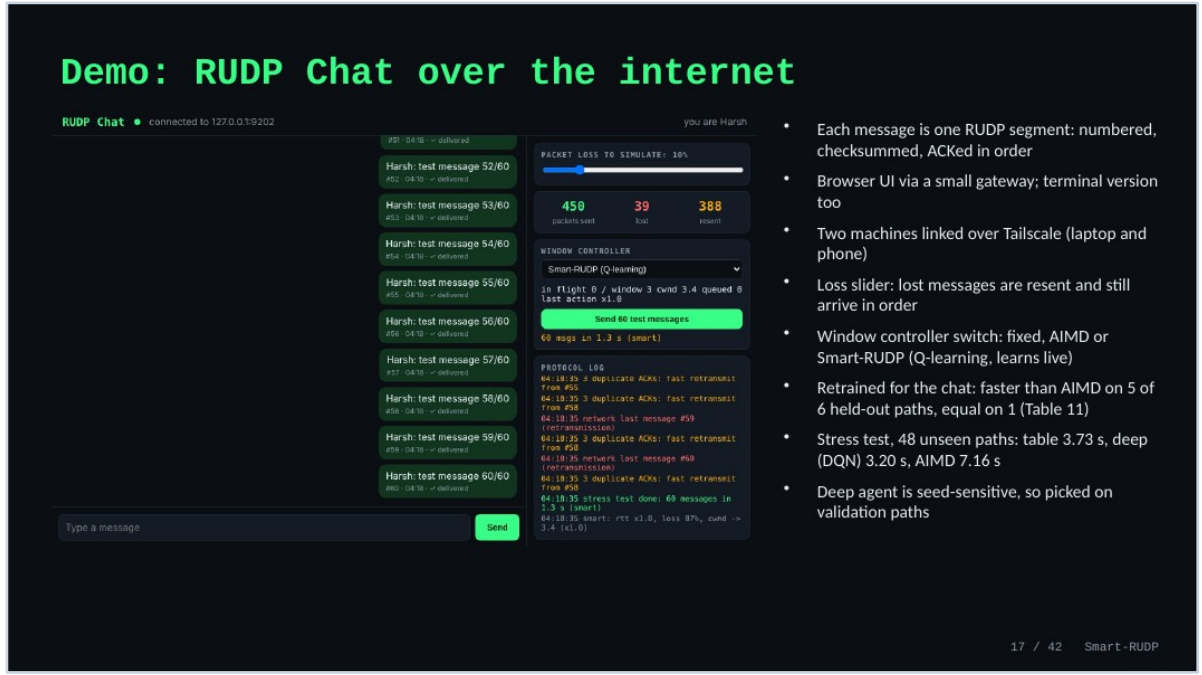
IF ASKED If Wireshark is open live: Analyze, Decode As, UDP 8888 = RUDP, then use these two filters.

SAY

- Left: the first three packets, SYN, SYN-ACK and ACK, decoded by our dissector (filter frame.number <= 3).
- Right: filter smartrudp.seq == N shows the same segment twice, a retransmission.

17. Demo: RUDP Chat over the internet

Time on this slide: 1:00



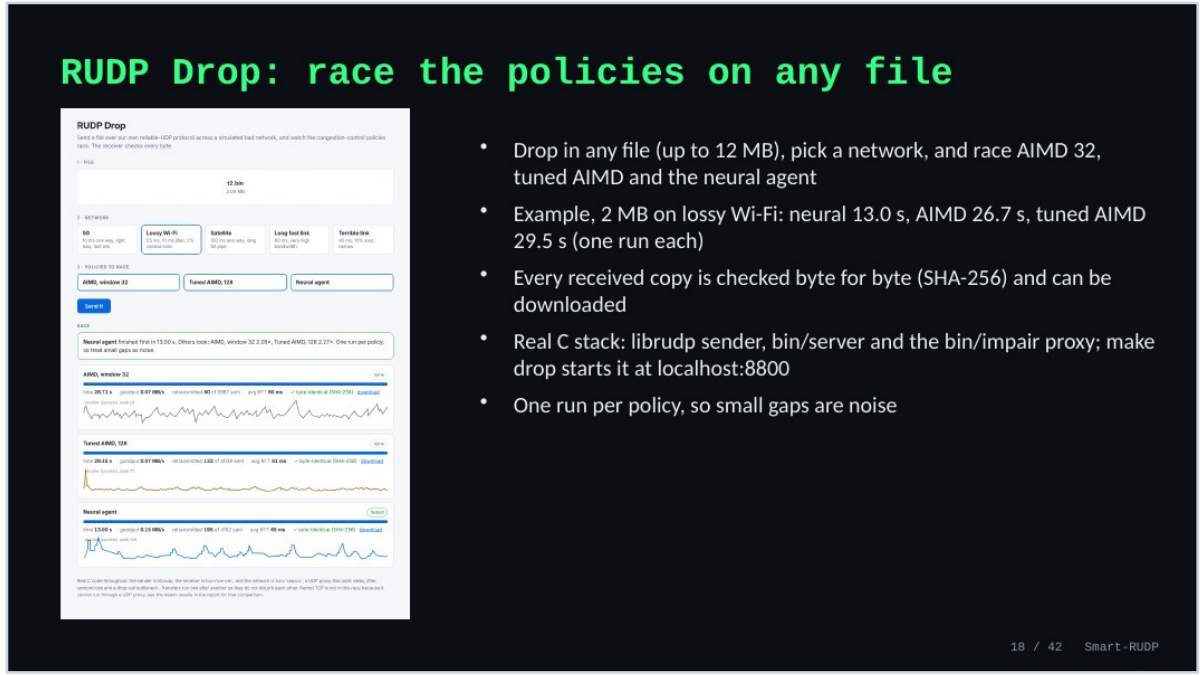
IF ASKED Q: Is that a fair win? A: Both start from the same window of 10; held-out paths and seeds; honest limits: simulated, and without the no-shrink rule the agent lost. Q: Did it win in Section 8? A: No, that result stands. Q: Where does it lose? A: Tight bottleneck queues and very high loss; the fixed window of 8 is hard to beat there. A broader-trained agent did worse, so we kept the first.

SAY

- The protocol in use: each message is one numbered, checksummed RUDP segment, ACKed in order.
- There is a terminal version and a browser version; two devices were linked over Tailscale, laptop and phone.
- Drag the loss slider: lost messages are resent and still arrive in order. The right panel shows sent, lost and resent counts and the log.
- There is a controller switch: fixed window, AIMD, or Smart-RUDP. For the chat we retrained Smart-RUDP on simulated paths: it was faster than AIMD on 5 of 6 held-out paths and equal on 1, because AIMD resets to one packet on random loss. Say it is simulated, and that the agent may not shrink the window without a congestion signal. Stress test: 48 random paths the agent never trained on, 3 seeds each. Mean delivery time was 3.73 seconds for Smart-RUDP against 7.16 for AIMD. Smart-RUDP was faster on 27 paths, slower on 5, within 5 percent on 16. So on our data, in simulation, it beats AIMD in the chat setting. There is also a deep neural-network option in the controller dropdown; it gave 3.20 seconds mean on the stress paths against 3.73 for the table.

18. RUDP Drop: race the policies on any file

Time on this slide: 1:00



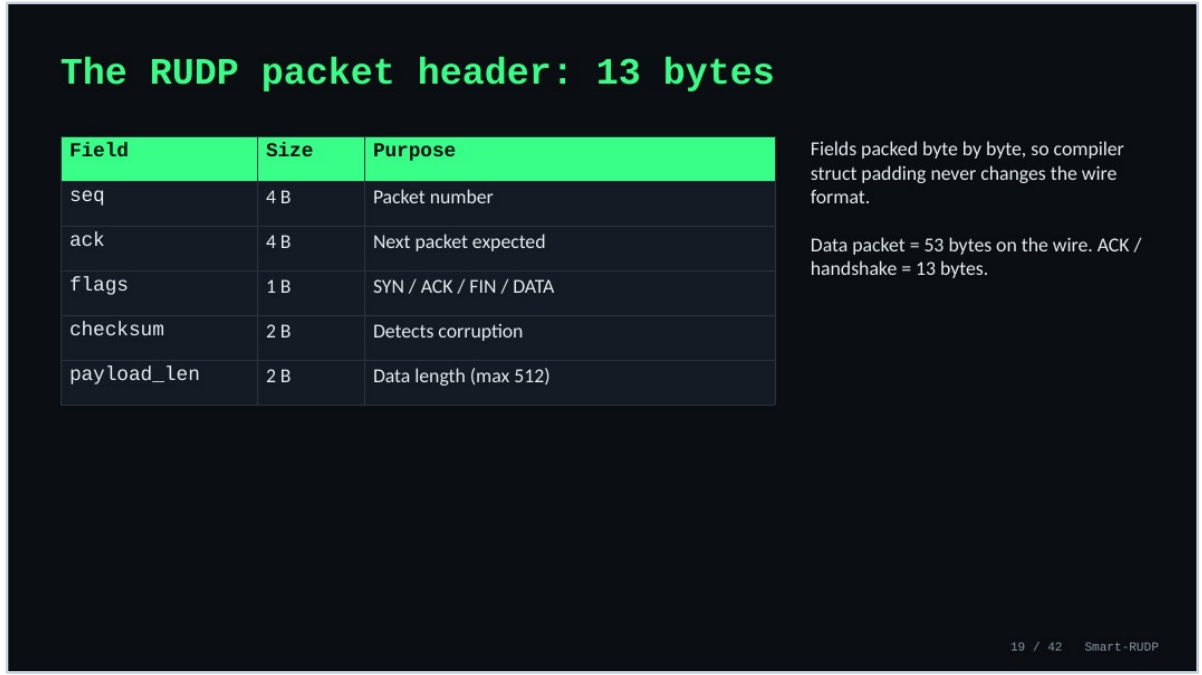
IF ASKED Is kernel TCP in the race? No, it cannot run through a UDP proxy. The kernel comparison is in the netem results.

SAY

- This is an application built on our library. You drop in any file, pick a network such as lossy Wi-Fi or satellite, and it sends the file through our real C stack with three policies, one after another.
- In this run a 2 MB file took 13 seconds with the neural agent and 27 and 29 seconds with the two AIMD settings. Every received copy is compared byte for byte, so you can see it is the same file.
- It is one run per policy, so small gaps are noise. The point is that the protocol is usable by an ordinary program through a small library.

19. The RUDP packet header: 13 bytes

Time on this slide: 0:40



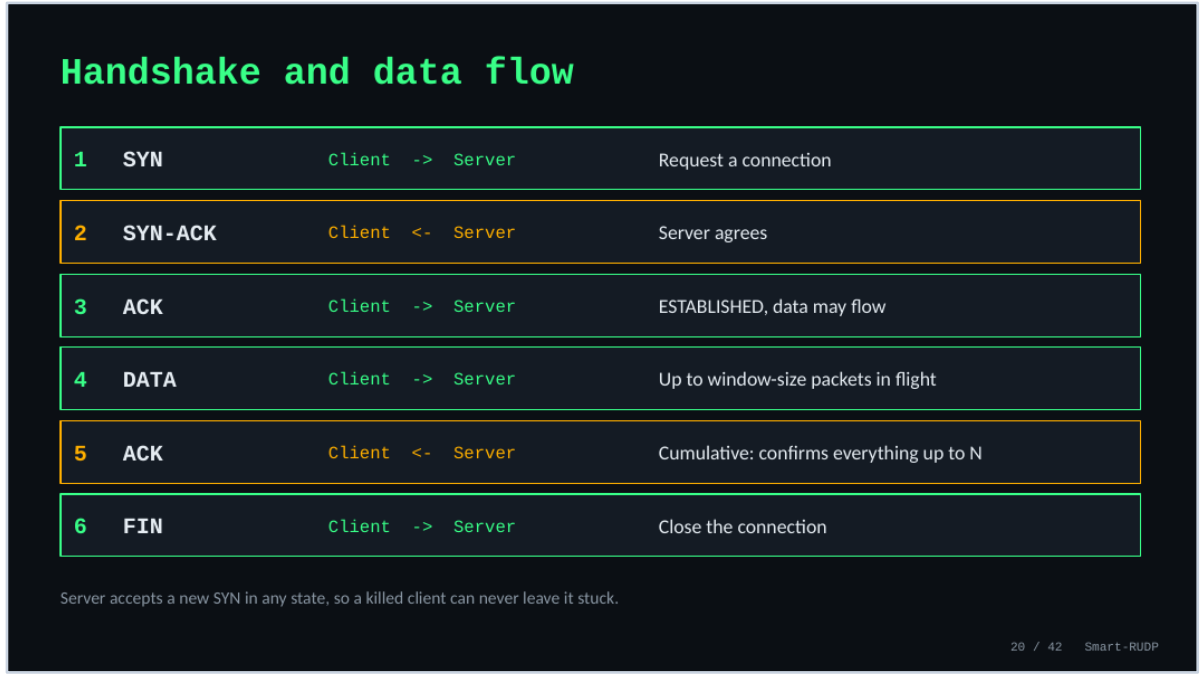
SAY

- 13-byte header: sequence 4, ack 4, flags 1, checksum 2, payload length 2.
- Fields are packed byte by byte so compiler padding never changes the wire format.
- A data packet is 53 bytes on the wire; ACKs and handshake packets are 13.

IF ASKED Q: How does the checksum work? A: 16-bit one's-complement sum over header and payload, like UDP/TCP. A corrupted packet fails and is recovered by retransmission.

20. Handshake and data flow

Time on this slide: 0:50



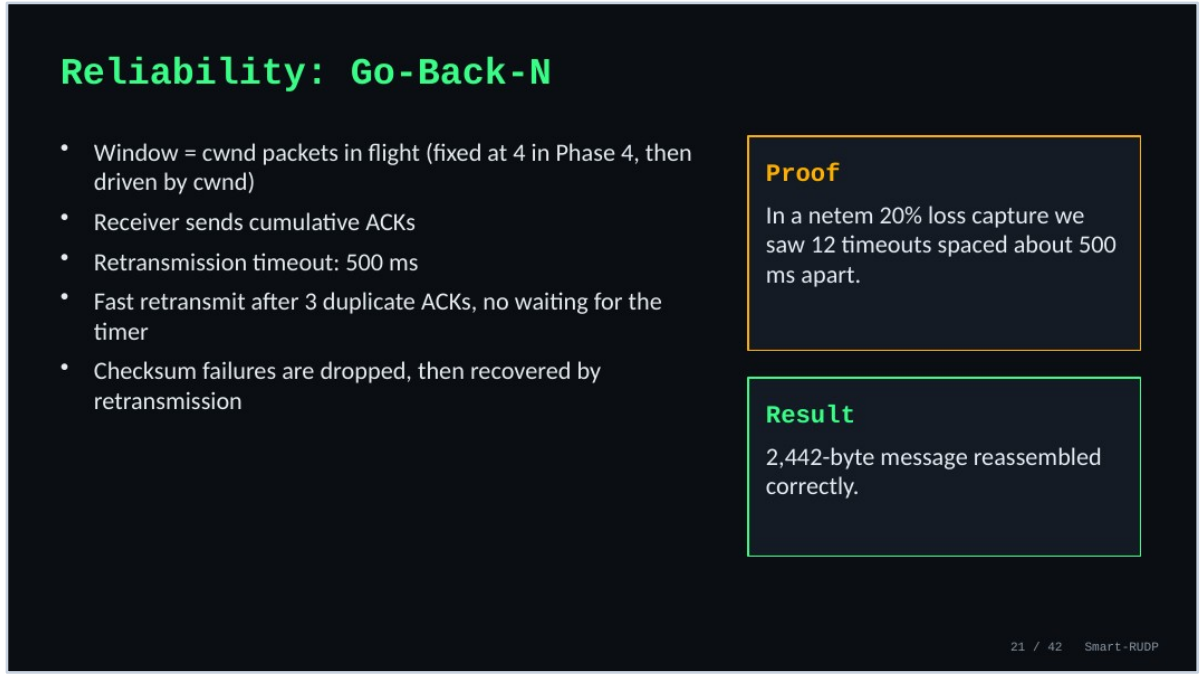
IF ASKED Q: What if the peer disappears? A: Sender keeps retransmitting on timeout. An abort after repeated timeouts is not implemented; it is the fix we would add.

SAY

- Three-way handshake: SYN, SYN-ACK, ACK. Then data, up to a window of packets in flight.
- ACKs are cumulative: ACK N confirms everything before N. FIN closes the connection.
- The server accepts a new SYN in any state, so a killed client can never leave it stuck. That was a real bug we fixed.

21. Reliability: Go-Back-N

Time on this slide: 0:50



IF ASKED Q: Why Go-Back-N? A: Simpler to build and explain. It wastes bandwidth under loss; Selective Repeat with SACK is our stated future work.

SAY

- The window is the number of packets in flight; it started fixed at 4 and is now driven by the congestion window.
- Timeout is 500 ms; fast retransmit triggers after 3 duplicate ACKs, no waiting for the timer.
- Proof: in a 20% loss netem capture we saw 12 timeouts about 500 ms apart. Result: the 2,442-byte message reassembled correctly.

22. Reliability: Selective Repeat (the default)

Time on this slide: 0:50



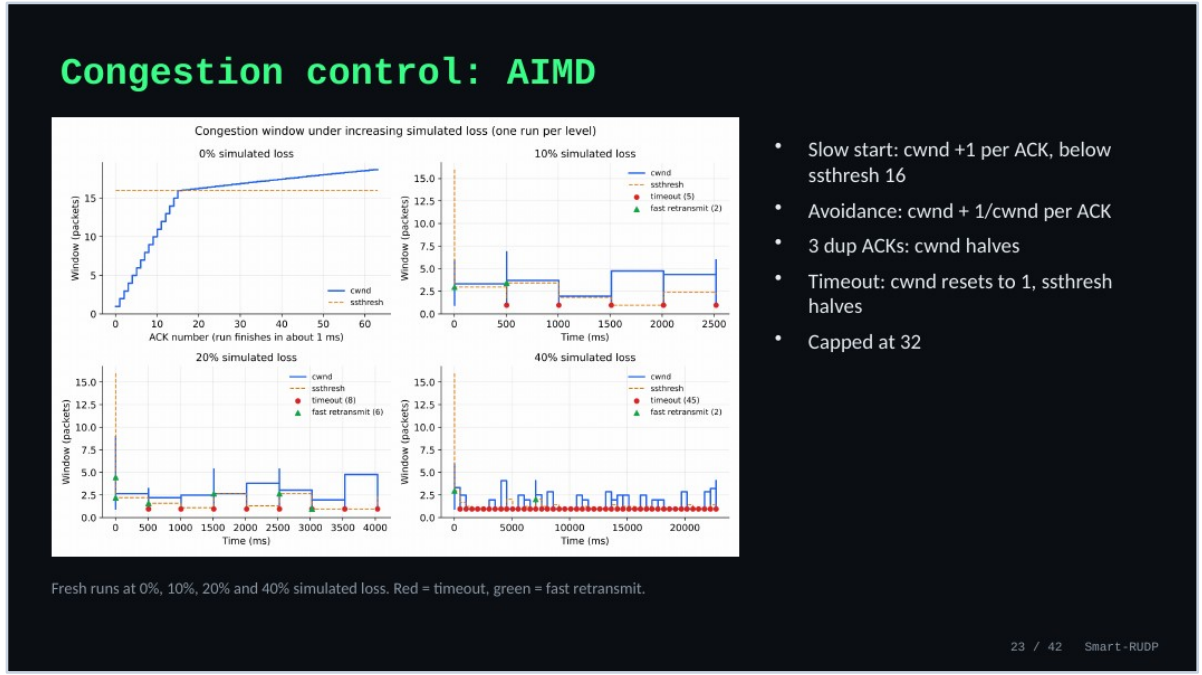
IF ASKED Is it textbook Selective Repeat? Nearly: per-packet timers and a buffering receiver, but we use a cumulative ACK plus a 64-bit bitmap instead of one ACK per packet, and the window is capped at 60.

SAY

- We swapped Go-Back-N for Selective Repeat as the default. The receiver now keeps out-of-order packets and tells the sender exactly which ones it holds in a bitmap inside every ACK; the sender has one timer per packet and resends only what is missing.
- The result: about ten times fewer retransmissions, and the neural agent's lead over AIMD grows to 46 percent on the lossy path and 50 percent on the satellite path.
- The 10 percent loss failure we found earlier was mostly a Go-Back-N effect: with Selective Repeat the agent ties AIMD there.
- Go-Back-N is still in the code behind a flag, and the earlier tables in the report were measured with it and are labelled that way.

23. Congestion control: AIMD

Time on this slide: 0:50



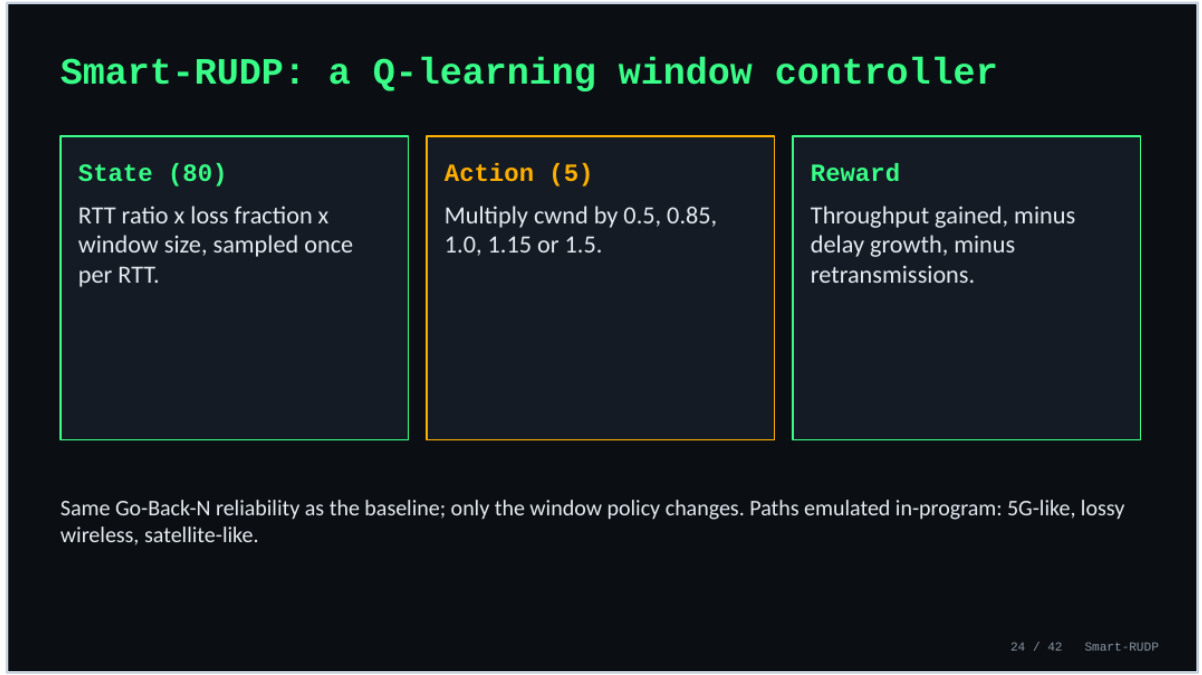
IF ASKED Q: Why cap 32 and ssthresh 16? A: The cap stops a lossless run growing without bound; 16 is where doubling switches to linear growth.

SAY

- Slow start: window grows by one per ACK until the threshold of 16. Then avoidance: plus 1/window per ACK.
- Three duplicate ACKs: window halves. A timeout: window resets to 1 and the threshold halves. Cap is 32.
- The graph shows fresh runs at 0, 10, 20 and 40 percent loss. Red marks are timeouts, green are fast retransmits.

24. Smart-RUDP: a Q-learning window controller

Time on this slide: 0:50



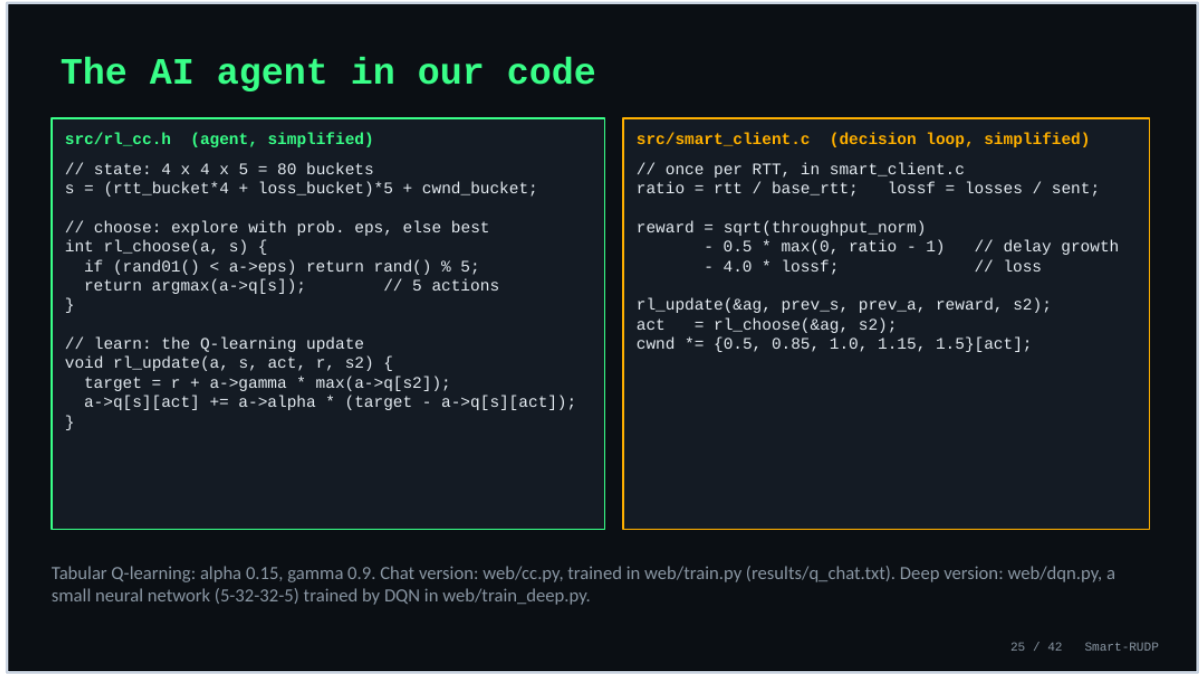
IF ASKED Q: Is it really learning? A: Yes, 40 training episodes; the learning-curve figure shows it. It is small and tabular with limited training.

SAY

- Q-learning with 80 states (RTT ratio, loss fraction, window size), sampled once per RTT.
- Five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5.
- Reward: throughput gained, minus delay growth, minus retransmissions. Same Go-Back-N reliability; only the window policy changes. Paths are emulated in the program: 5G-like, lossy wireless, satellite-like.

25. The AI agent in our code

Time on this slide: 1:00



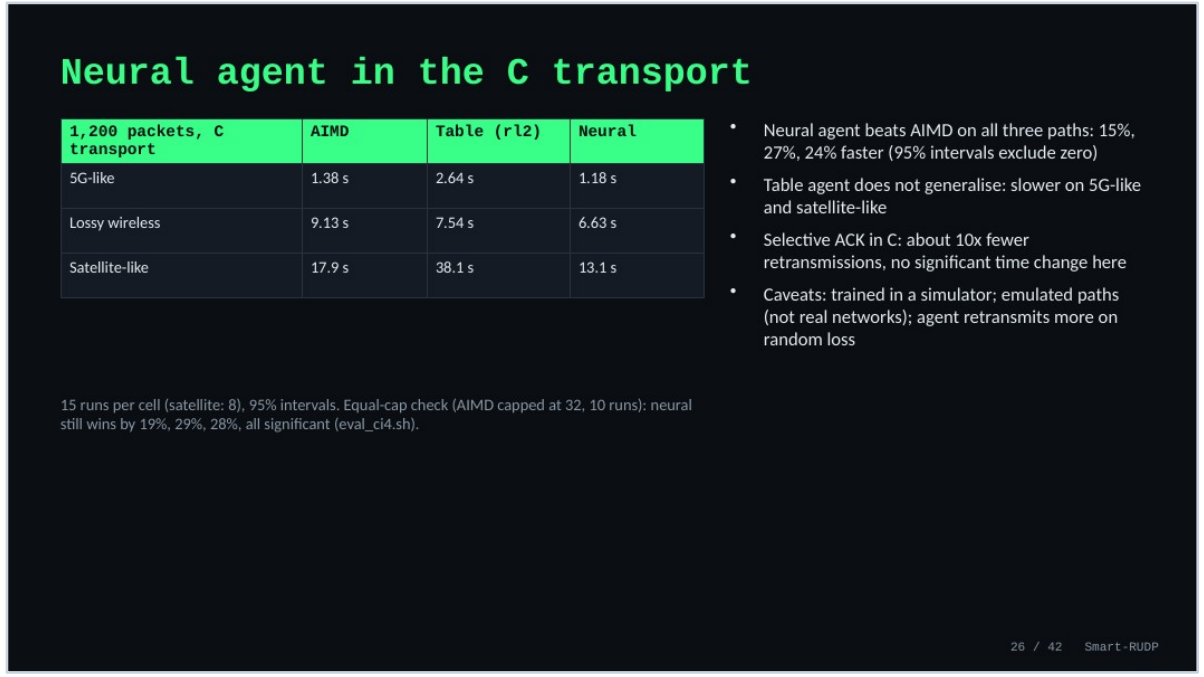
IF ASKED Q: Is this deep RL? A: The C agent is tabular. The chat also has a deep agent, a small DQN in numpy (5-32-32-5). Q: Does deep beat the table? A: Modestly: 3.20 s vs 3.73 s mean on 48 unseen paths, faster on 19, slower on 6. Q: Any problem? A: Training is seed-sensitive: 1 of 4 seeds failed, so we select on validation paths. It is simulated and not in the C code. Q: Where is it trained? A: web/train.py for the chat, the smart client trains online for the main experiments; table in results/q_chat.txt.

SAY

- This is the agent itself. Left: rl_cc.h, 73 lines of C. The state is 80 buckets of RTT ratio, loss and window size.
- choose picks one of five window multipliers, usually the best, sometimes random so it keeps exploring.
- update is the Q-learning rule: move the value of the action we took toward the reward plus the best value of the next state.
- Right: the loop in smart_client.c. Once per RTT it computes the reward, throughput minus delay growth minus loss, updates the table, picks the next action and scales the window.
- The code on the slide is simplified; the real files are in the repo.
- We also built a deep version: web/dqn.py is a small neural network, two hidden layers of 32, trained with DQN in the same simulator. It reads continuous inputs instead of buckets.

26. Neural agent in the C transport

Time on this slide: 1:00



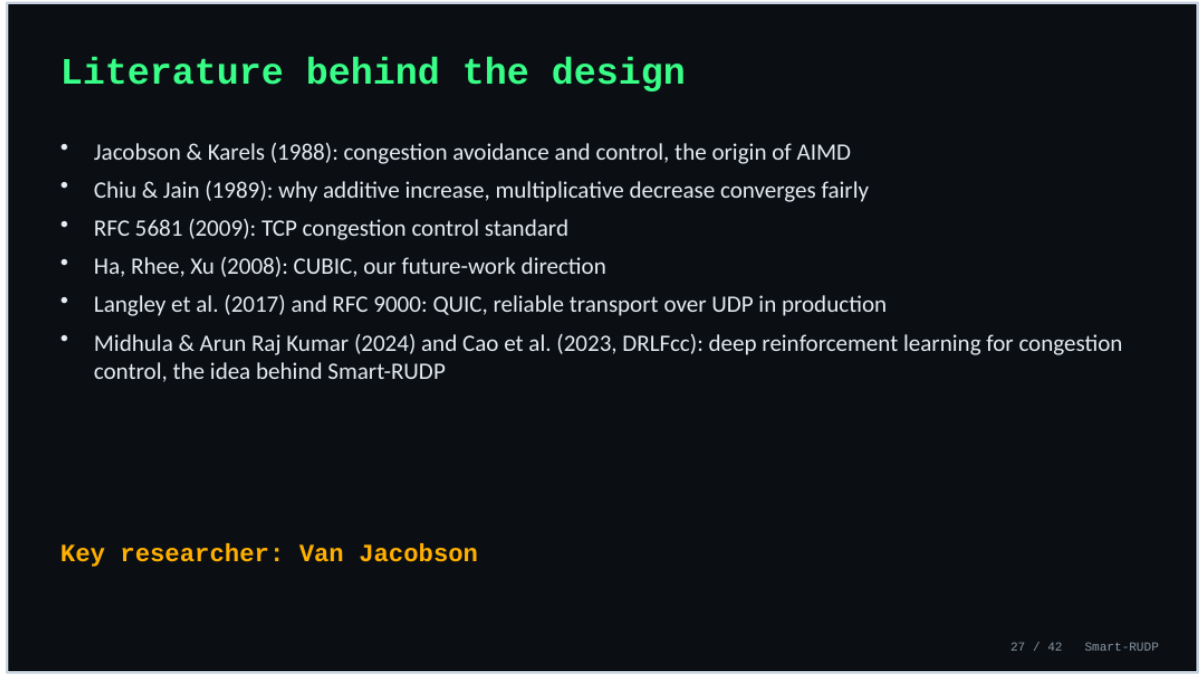
IF ASKED Q: So Smart-RUDP now beats AIMD? A: The neural version does, in the C transport on emulated paths, trained in a simulator; the original table agent in Section 8 did not. Q: Is it fair? A: Same start window; AIMD keeps a cap of 128 while the agents are capped at 32; the agent retransmits more on random loss. Q: Why no gain from SACK? A: Those paths are limited by the bottleneck rate, so extra retransmissions cost little; the simulator, with different paths, showed a large gain.

SAY

- We then put the neural agent into the C transport, the same code as Section 8, and tested it on the same three emulated paths with 15 runs each.
- AIMD and the agents all start from a window of 10. The neural agent was faster than AIMD on all three: 15 percent on 5G-like, 27 percent on lossy, 24 percent on satellite-like. The intervals exclude zero.
- The retrained table did not carry over: it was slower on 5G and satellite. A network generalises to unseen paths where a table does not.
- We also added selective acknowledgments to the C code: about ten times fewer retransmissions, but no significant change in completion time on these emulated paths.
- We also reran it with AIMD capped at 32 like the agent: the neural agent still wins by 19, 29 and 28 percent, all significant.

27. Literature behind the design

Time on this slide: 0:30



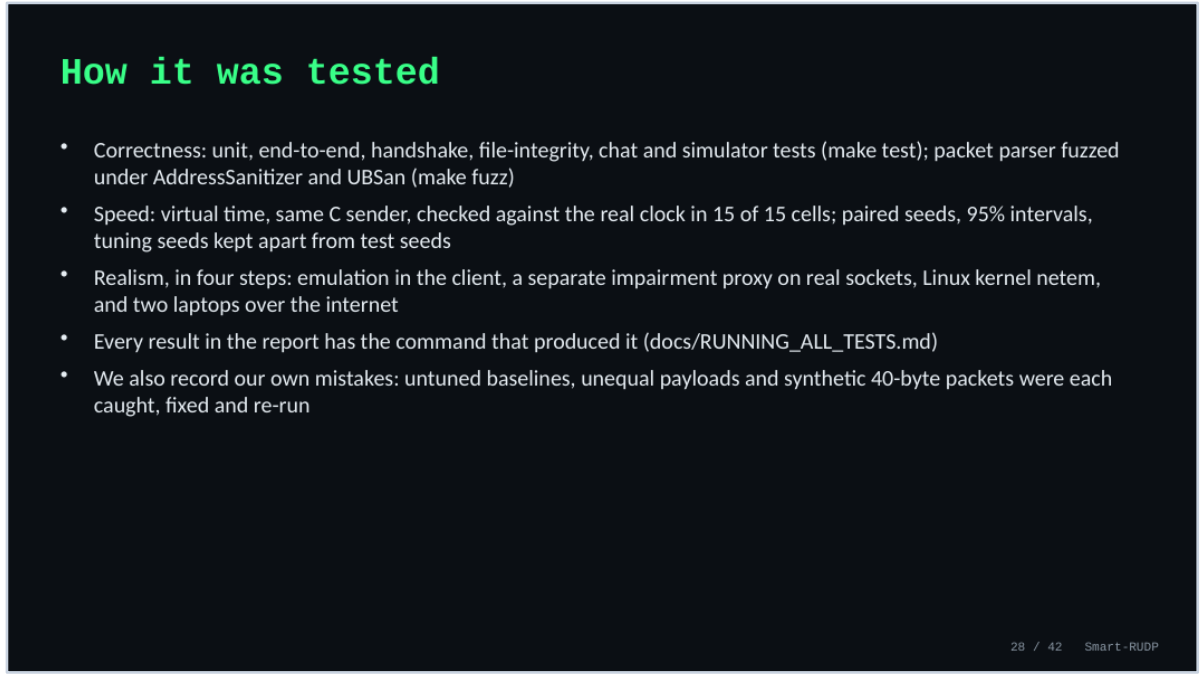
IF ASKED Q: Why AIMD? A: Chiu and Jain show it converges to a fair share. Q: Are those papers deep RL and yours is not? A: Yes; we used a lookup table, which is simpler and inspectable.

SAY

- Jacobson and Karels 1988 is the origin of AIMD; Chiu and Jain 1989 explains why it converges fairly.
- RFC 5681 is the standard; CUBIC is future work; QUIC and RFC 9000 are reliable transport over UDP in production.
- The AI part comes from Midhula and Arun Raj Kumar (2024) and Cao et al. (2023, DRLFcc): deep reinforcement learning for congestion control. Smart-RUDP applies the same idea with a smaller tabular Q-learning agent.
- Key researcher: Van Jacobson.

28. How it was tested

Time on this slide: 0:40



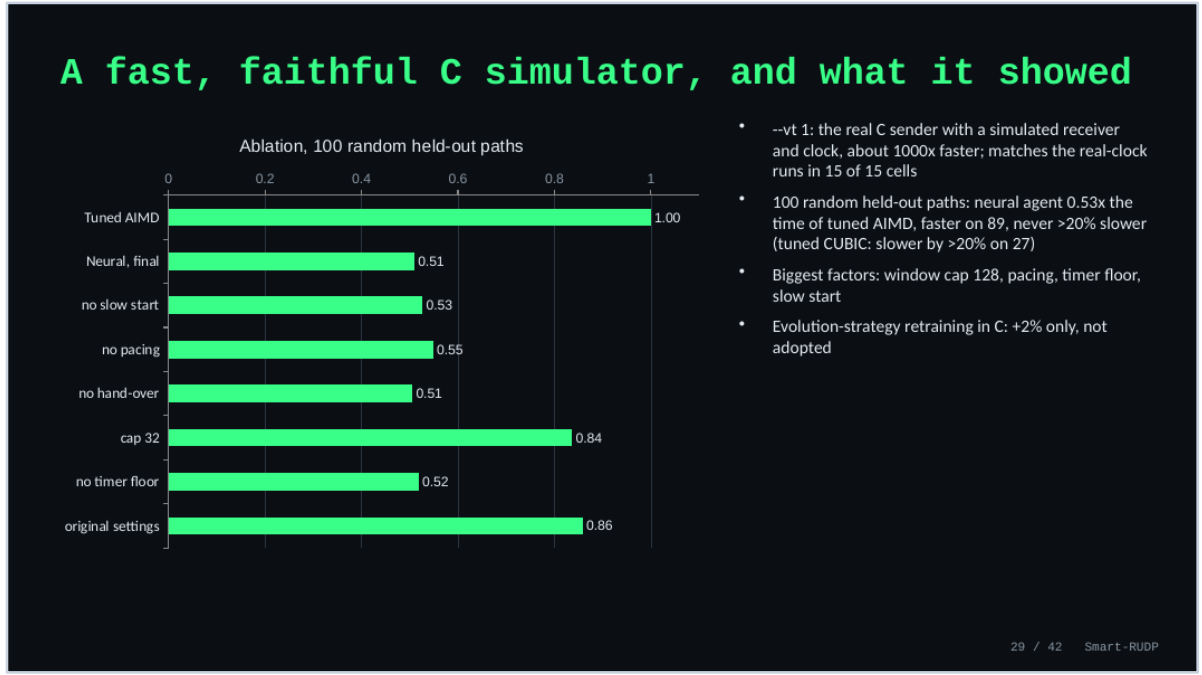
IF ASKED Did you make mistakes? Yes, and we logged them: untuned baselines, unequal payloads, and synthetic 40-byte packets in the kernel test, each found, fixed and re-run.

SAY

- Correctness first: unit, end-to-end, handshake, file-integrity, chat and simulator tests, and the packet parser is fuzzed under AddressSanitizer.
- For speed we use virtual time, which is the same C sender on a simulated clock. We checked it against the real clock, and it agreed in all 15 of 15 cells. We use paired seeds with 95 percent intervals and keep tuning seeds apart from test seeds.
- Realism comes in four steps: emulation in the client, a separate proxy on real sockets, the Linux kernel with netem, and two laptops over the internet. Every result has its command in the report.

29. A fast, faithful C simulator, and what it showed

Time on this slide: 1:00



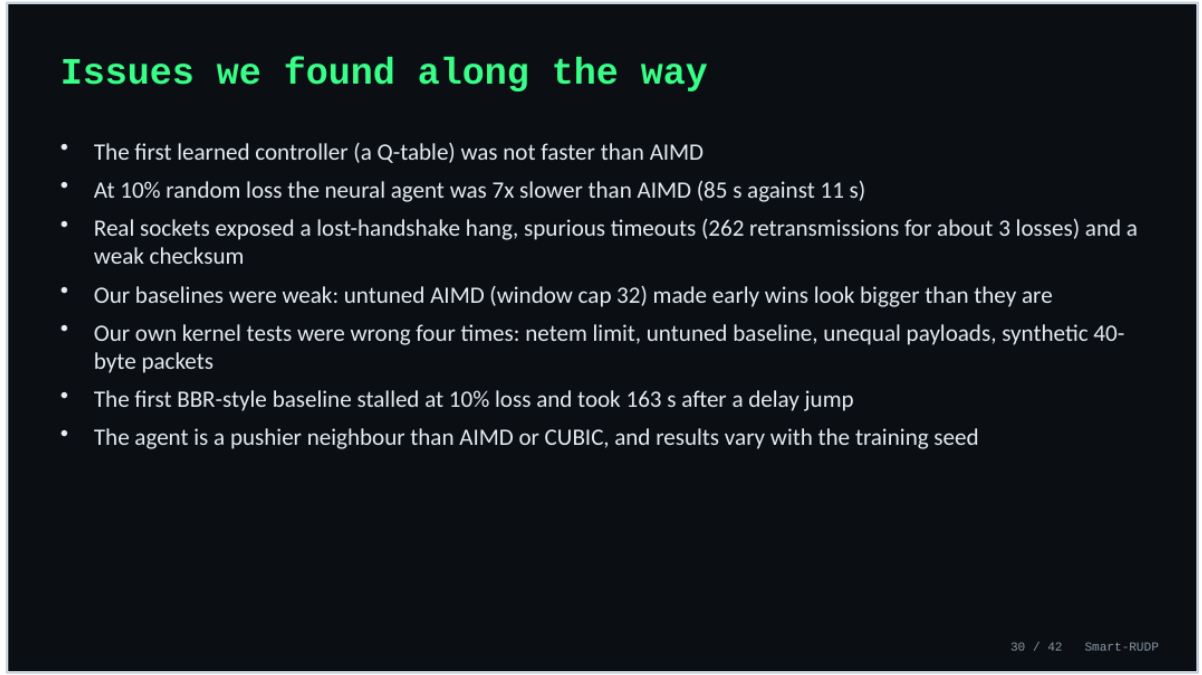
IF ASKED Is the test set separate from tuning? Yes. Slow-start and hand-over settings were chosen on a different set of random paths; the 100 test paths were used only for the reported numbers.

SAY

- Our old simulator was in Python and never matched the C code exactly. So we added a virtual-time mode to the C sender itself: same code, simulated receiver and clock, about a thousand times faster. It reproduces the real-clock results in all 15 calibration cells.
- That let us test on 100 random paths that we never tuned on. The neural agent needs 0.53 times the time of tuned AIMD, is faster on 89 paths and never more than 20 percent slower. Tuned CUBIC is slower by more than 20 percent on 27 paths.
- The ablation shows which parts matter: the window cap of 128 most, then pacing, the timer floor and slow start. We also tried evolution-strategy retraining directly in the C sender and it only gave 2 percent, so we did not adopt it.

30. Issues we found along the way

Time on this slide: 0:50



IF ASKED How did you find these? By testing on real sockets and the kernel, comparing against tuned baselines, and checking our own numbers, for example a goodput above the link rate gave away the 40-byte packet bug.

SAY

- Now the issues, and we list them plainly. Our first learned controller, a Q-table, was not faster than AIMD. At 10 percent random loss the neural agent was 7 times slower, 85 seconds against 11.
- Real sockets exposed a lost-handshake hang, spurious timeouts, 262 retransmissions for about 3 losses, and a weak checksum. Our early baselines were weak: untuned AIMD made the first wins look bigger than they are.
- Our own kernel tests were wrong four times and our first BBR-style baseline had two bugs. And the agent is a pushier neighbour than AIMD or CUBIC. Each of these has a fix on the next slide.

31. What we changed to make it better

Time on this slide: 1:00



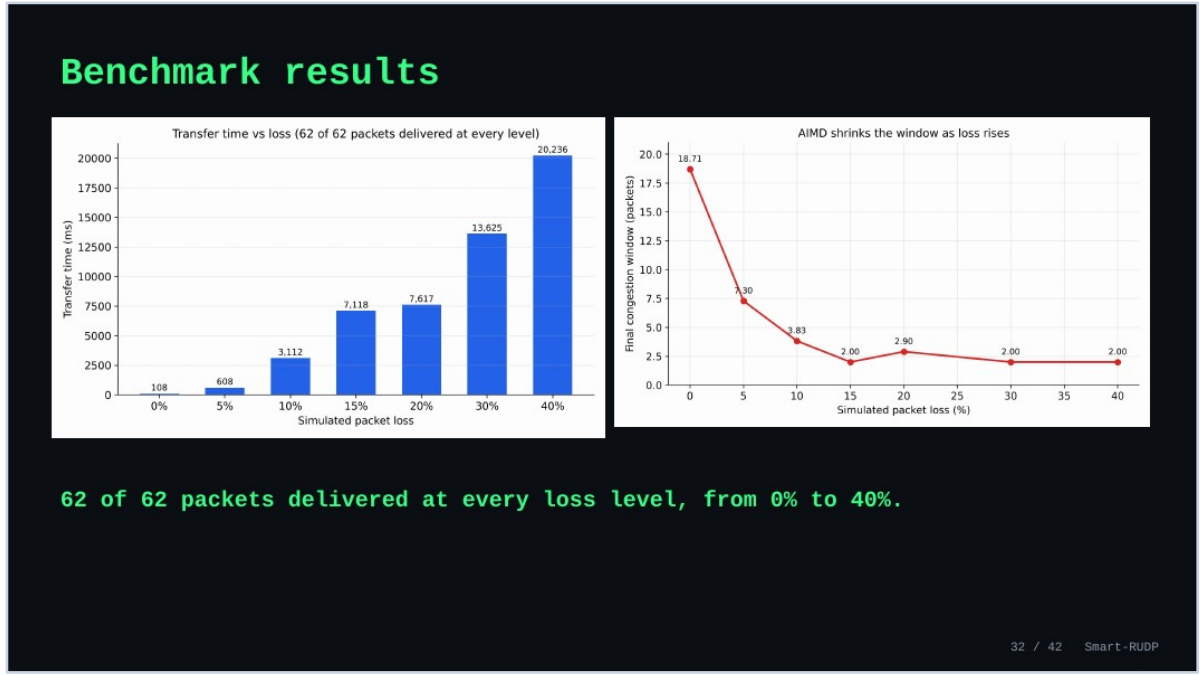
IF ASKED Which change mattered most? The ablation says window cap 128, pacing, the timer floor and slow start do most of the work, and the hand-over is what makes the hostile path safe.

SAY

- Every issue led to a change. Go-Back-N with a fixed timer became Selective Repeat with bitmap ACKs, per-packet timers and an adaptive timer, about 10 times fewer retransmissions. The byte-sum checksum became CRC-16.
- The lost-handshake hang was fixed as TCP does it. The Q-table became a small neural network with a bigger window cap, slow start and pacing. The 10 percent loss collapse is handled by handing over to AIMD while loss is high: 13.9 seconds instead of 84.9.
- We tuned the AIMD and CUBIC baselines so every win is against a fair opponent, and we fixed our test methodology and re-ran the kernel tests.

32. Benchmark results

Time on this slide: 0:40



IF ASKED Q: Single runs? A: Yes, the original benchmark ran each level once. Smart-RUDP evaluation uses 6 runs per scenario.

SAY

- 62 of 62 packets were delivered at every loss level from 0 to 40 percent.
- Higher loss costs time and retransmissions, never correctness.

33. Problems we hit and fixed

Time on this slide: 0:40



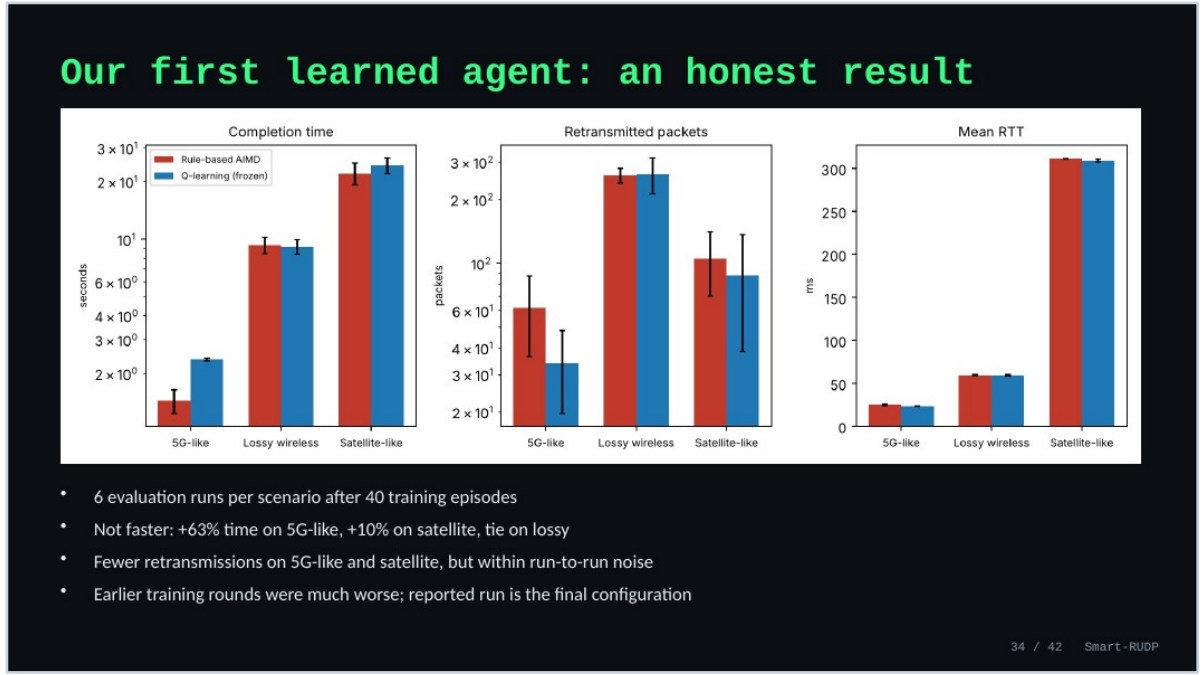
IF ASKED Say these quickly: it shows real debugging, which is what separates a project from a lab.

SAY

- Server stuck after a killed client: it hung 15.5 s; fix was to accept SYN in any state.
- A leftover netem rule distorted results; we now check tc qdisc before each test.
- Windows Wireshark saw nothing: capture inside WSL.
- Benchmark script portability: replaced date and grep tricks with awk and sed.

34. Our first learned agent: an honest result

Time on this slide: 0:50



IF ASKED Q: Then what was the point? A: The experiment. We tested whether a learned window helps and report the answer, including the negative result.

SAY

- Be honest: it did not beat AIMD. Plus 63 percent time on 5G-like, plus 10 percent on satellite, a tie on the lossy path.
- It used somewhat fewer retransmissions on two paths, within run-to-run noise.
- Earlier training rounds were much worse; this is the final configuration. The finding is that a simple learner does not beat a tuned rule.
- In the chat, retrained for bursty random-loss traffic, it does beat AIMD on most simulated paths; mention that on slide 19.

35. More evidence: baselines, SACK, reordering

Time on this slide: 1:00



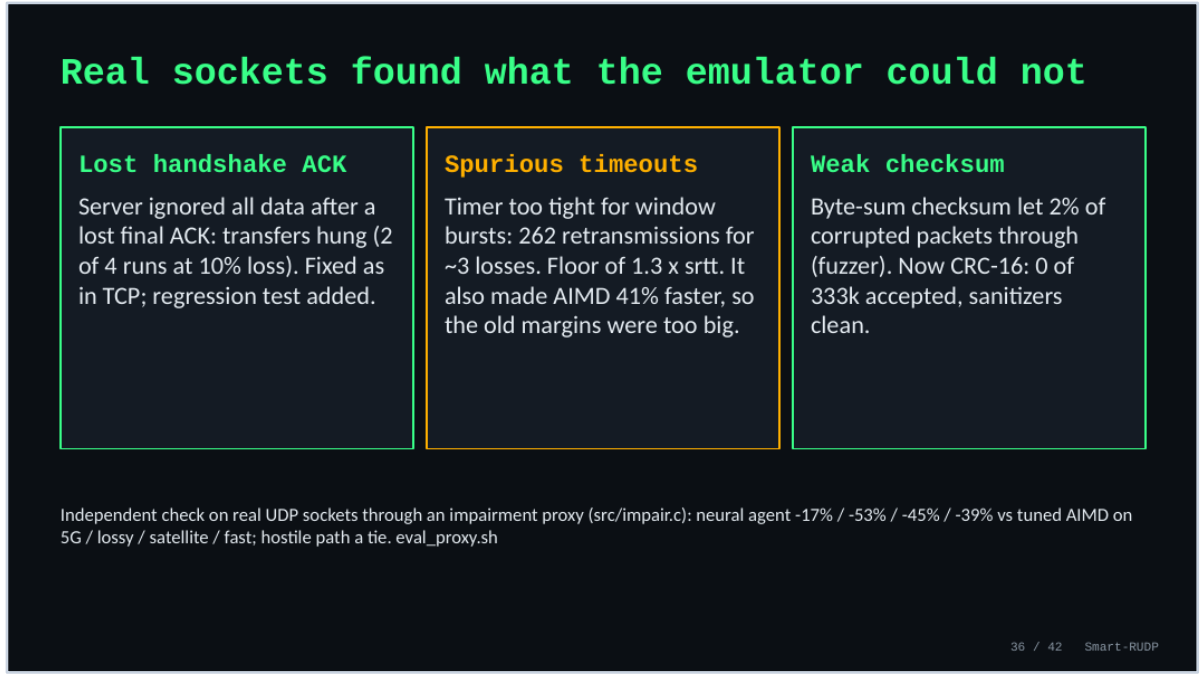
IF ASKED Q: Is the TCP row real TCP? A: No, it is a model with the same window cap; real kernel TCP needs netem on Linux, and we provide the script. Q: Why did you not implement SACK in C? A: Time; the simulation shows it is the biggest win and it is the first item of future work. Q: Is 20 runs enough? A: The intervals exclude zero for 5G and satellite and include zero for lossy. Did you check it is not a simulator artefact? Yes: with the realistic rule the simulated gain mostly disappears, so we lean on the C results, which use that rule and equal window caps.

SAY

- The abstract also promised TCP and plain UDP baselines, so we ran them on the same 48 unseen simulated paths.
- Plain UDP delivered only 68 percent of the messages, so reliability is needed. Our TCP model, with selective acknowledgments, has the lowest median at 1.48 seconds.
- Smart-RUDP deep is at 3.20 seconds mean and is about even with the TCP model path by path: faster on 22, slower on 23.
- The gap to TCP is mostly Go-Back-N. A selective-ack version of our transport cut AIMD from 7.16 to 4.90 seconds in simulation.
- Reordering hurts Go-Back-N badly: AIMD goes from 0.82 to 6.79 seconds at 20 milliseconds of jitter. The learned agents cope better and selective ack removes most of the damage.
- For the C experiments we repeated 20 runs per mode: Smart-RUDP is still slower on 5G and satellite and ties on lossy.
- Be upfront: our simulator originally allowed a duplicate-ACK retransmission storm that flatters aggressive controllers. With the realistic rule the neural agent is about level with AIMD in simulation (7.27 s against 7.75 s). The C results are the evidence we rely on.

36. Real sockets found what the emulator could not

Time on this slide: 0:50



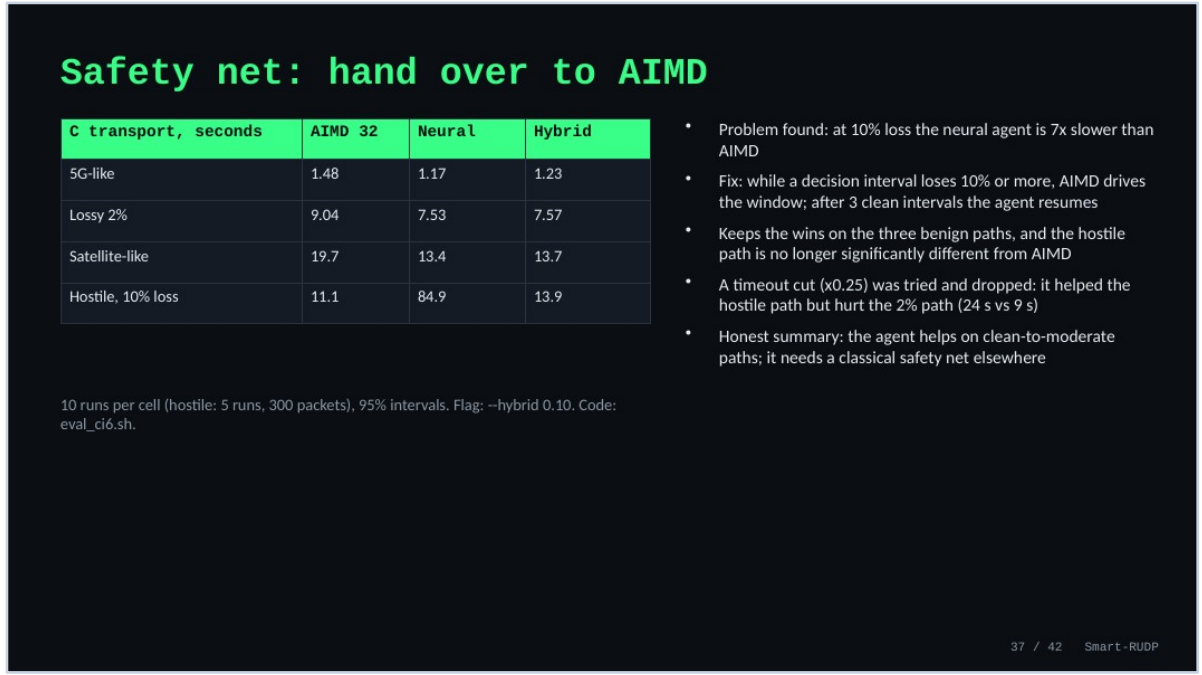
IF ASKED Is this a real network? It is real UDP sockets on one machine, with the impairment applied by a separate process. It is not a kernel netem or a multi-machine path; we provide the script for that but had no root access.

SAY

- The emulator is inside our client, so we also tested through a separate impairment process on real UDP sockets. That found three real defects.
- First, if the client's last handshake ACK is lost, the server ignored all following data and the transfer hung. We fixed it the way TCP does and added a regression test. Second, the spurious timeouts I just mentioned. Third, a fuzzer showed our byte-sum checksum let about 2 percent of corrupted packets through, so it is now CRC-16 and the fuzzer accepts none.
- On real sockets the neural agent is again fastest on every path: 17, 53, 45 and 39 percent faster than tuned AIMD on 5G, lossy, satellite and fast, and a tie on the hostile path.

37. Safety net: hand over to AIMD

Time on this slide: 0:50



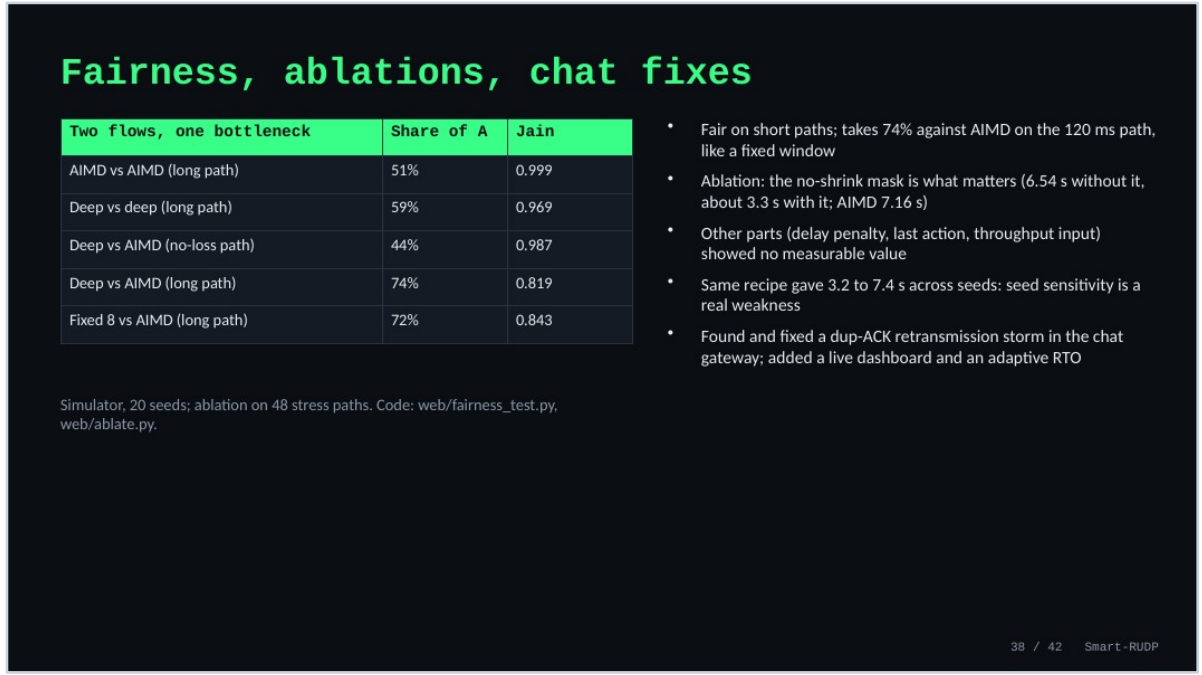
IF ASKED Is the hybrid still 'AI'? Yes on clean and moderate paths, where the neural network decides; on heavy loss it deliberately defers to AIMD. It is a rule we added, not something the network learned.

SAY

- While checking our own result we found where the neural agent fails: on a hostile path with 10 percent loss it needed about 85 seconds where AIMD needed 11.
- The fix is a hand-over: when a decision interval loses 10 percent or more of its packets, plain AIMD controls the window; after three clean intervals the agent takes over again.
- With that, the three benign paths keep the agent's advantage and the hostile path is back near AIMD at about 14 seconds.
- We also tried cutting the window on every timeout; it fixed the hostile case but made the 2 percent path much worse, so we dropped it. So the learned agent is useful on clean to moderate paths and needs a classical safety net elsewhere.

38. Fairness, ablations, chat fixes

Time on this slide: 0:50



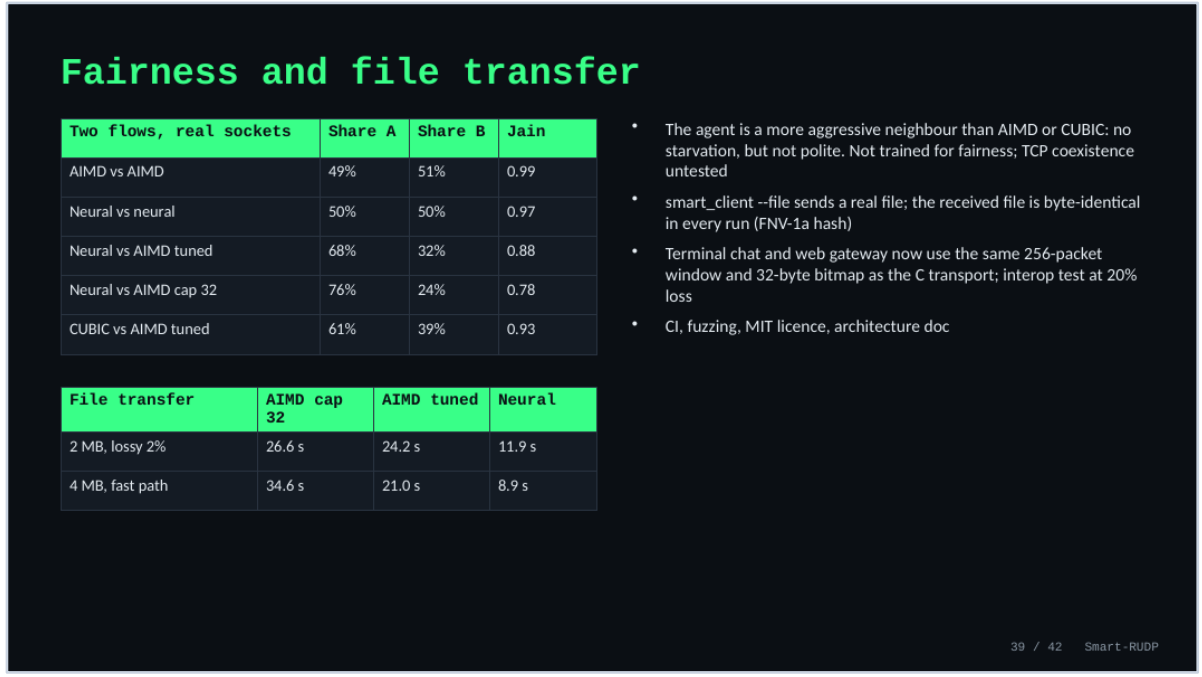
IF ASKED Is the agent fair? Mostly, but not on the long path against AIMD (74% share, Jain 0.82). Fairness was only tested in the simulator, not in C.

SAY

- We also tested two things people ask about. Fairness: two flows through one bottleneck. The neural agent is fair against itself and on short paths, but takes 74 percent against AIMD on the long 120 ms path, about like a fixed window.
- Ablation: we retrained the agent with one piece removed. Only the no-shrink mask matters; without it the agent is no better than AIMD. The other pieces showed no measurable value.
- The same recipe also gave very different results by seed, between 3.2 and 7.4 seconds, so we pick the seed on validation paths.
- While building the dashboard we found a retransmission storm in the web gateway and fixed it; there is also an adaptive timeout option now.

39. Fairness and file transfer

Time on this slide: 0:50



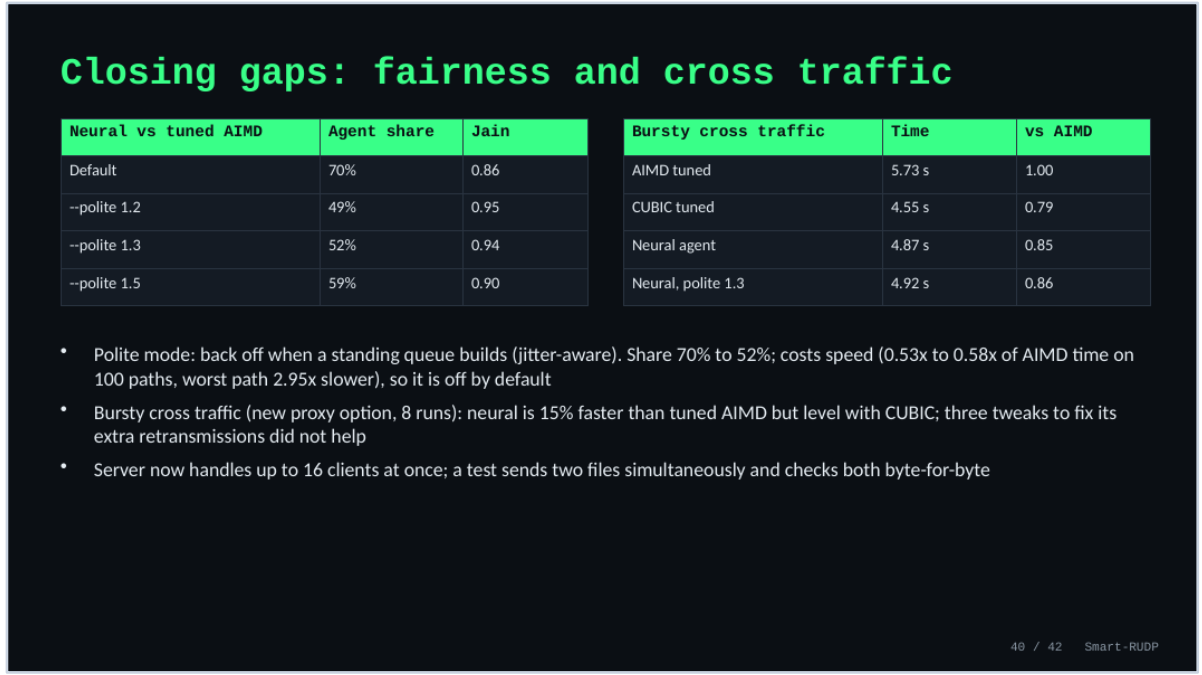
IF ASKED Would this be safe to deploy? Not on a shared link yet: it needs fairness-aware training and TCP coexistence tests. For a dedicated path it is a clear win.

SAY

- Fairness on real sockets: two neural flows split the link evenly, and so do two AIMD flows. But against tuned AIMD the neural agent takes 68 percent of the link, and 76 percent against the original AIMD. CUBIC takes 61 percent. So the agent is a more aggressive neighbour. Nothing starves, but it is not polite, and we did not test coexistence with TCP. That is our main limitation.
- We also added file transfer: the sender sends a real file and the received file is byte-identical in every run. A 4 megabyte file over the fast path takes 8.9 seconds with the agent against 21 seconds with tuned AIMD.
- The chat and web gateway now use the same window and bitmap as the C transport, and there is CI, fuzzing and an architecture document.

40. Closing gaps: fairness and cross traffic

Time on this slide: 0:50



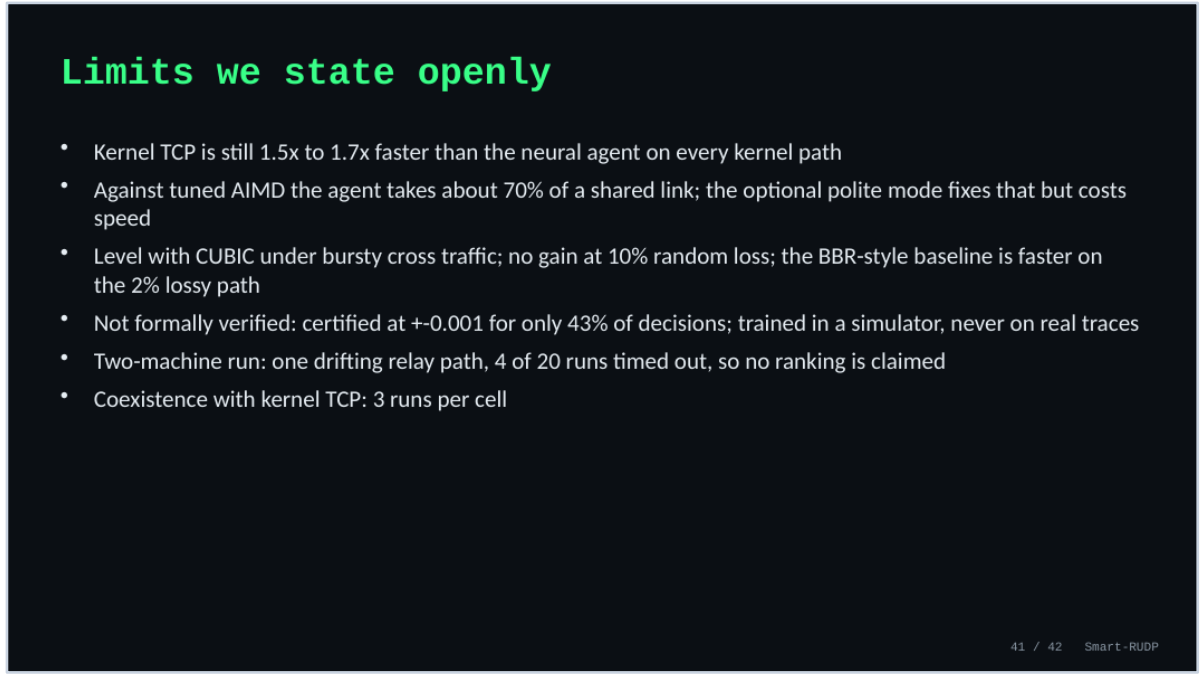
IF ASKED Is polite mode learned? No. It is a hand-made rule on the delay signal. A learned version needs a two-flow training environment, which is future work.

SAY

- We worked on the weaknesses we had listed. For fairness we added an optional polite mode: when a standing queue builds, the agent backs off. Over 8 runs against tuned AIMD it takes 52 percent of the link instead of 70, and the Jain index rises from 0.86 to 0.94. The price is speed: on 100 random paths the average time goes from 0.53 to 0.58 times AIMD's, and the worst path is 2.95 times slower, so it stays off by default.
- We also added bursty cross traffic to the test proxy. Here the result is less flattering: the agent is 15 percent faster than tuned AIMD, but level with CUBIC, and it retransmits about three times as often as AIMD. Three tweaks did not fix that, so we report it as a limit. Finally the server now handles up to 16 clients, checked by sending two files at once.

41. Limits we state openly

Time on this slide: 0:50



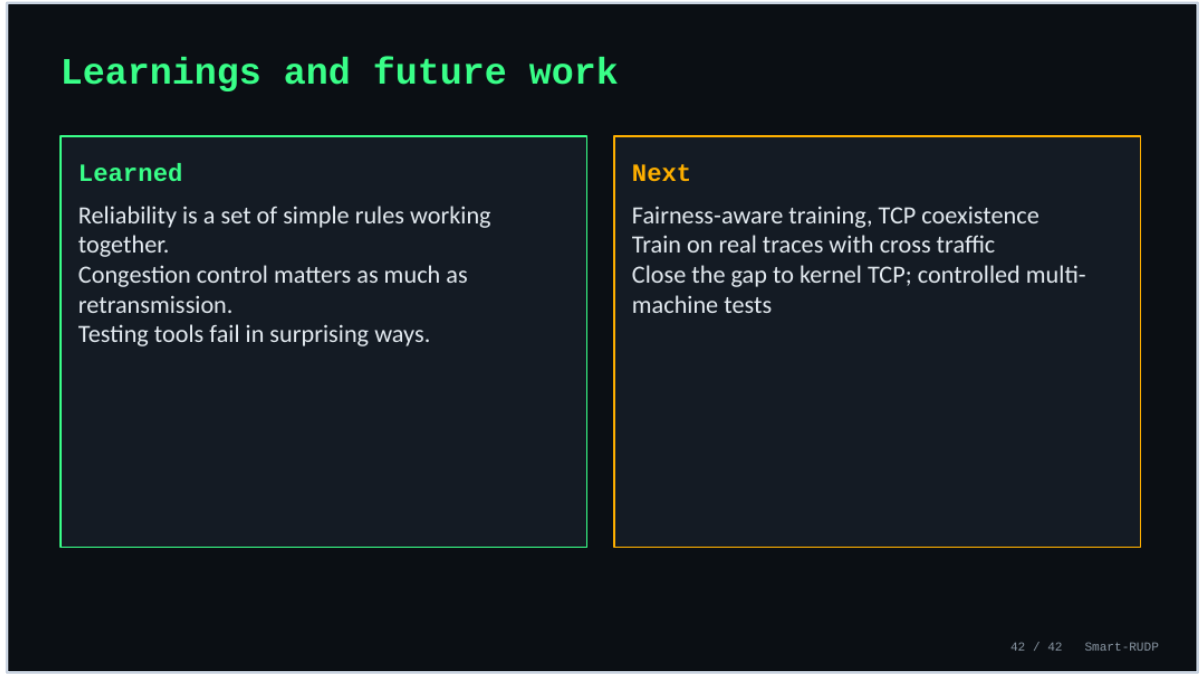
IF ASKED What would you do next? A direct-network multi-machine run with more repetitions, training with a competing flow so fairness is learned, and training on real traces.

SAY

- What is still not good. Kernel TCP is 1.5 to 1.7 times faster than our agent. Against tuned AIMD the agent takes about 70 percent of a shared link, and the polite mode that fixes it costs speed.
- It is level with CUBIC under bursty cross traffic, it gains nothing at 10 percent random loss, and the BBR-style baseline is faster on the 2 percent lossy path.
- It is not formally verified: only 43 percent of decisions are certified at plus or minus 0.001. It was trained in a simulator. The two-machine run was one drifting relay path with four timeouts, so we claim no ranking from it.

42. Learnings and future work

Time on this slide: 0:50



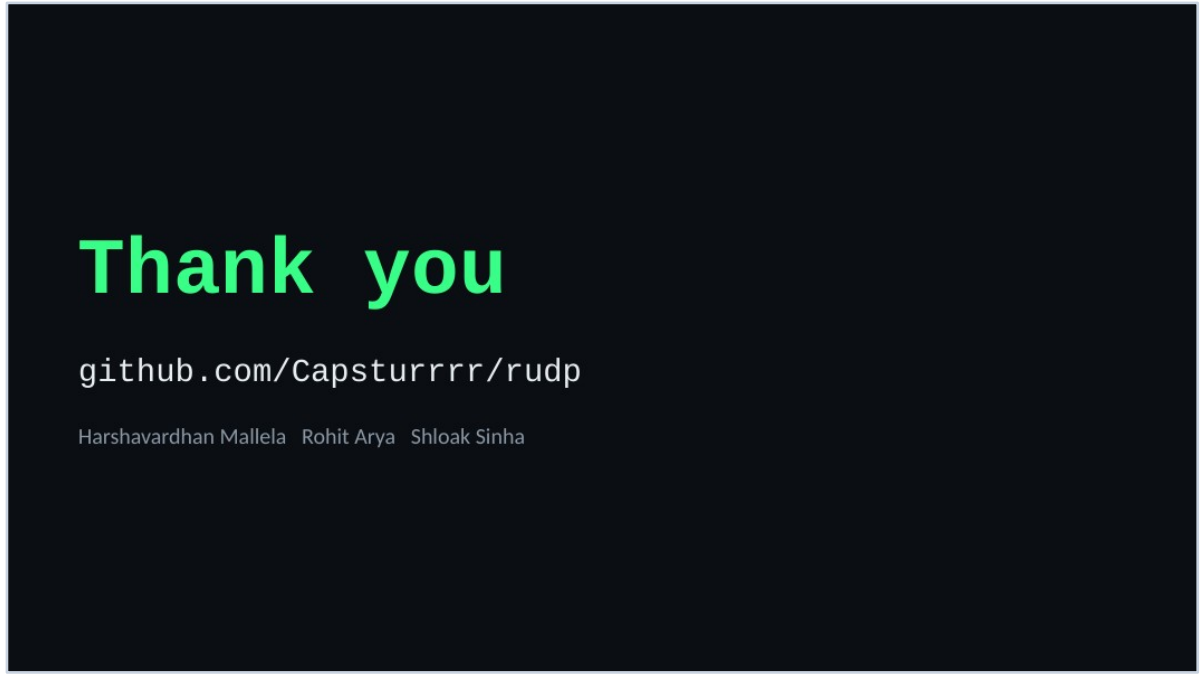
IF ASKED What would you do first? Run the netem script on a Linux machine for kernel TCP and RUDP on the same path, then add a competing flow to the training.

SAY

- What we learned: reliability is a set of simple rules working together, congestion control matters as much as retransmission, and tests tell you things about your own results that you do not want to hear.
- The limits are on the previous slide; in short: only one noisy two-machine run, the agent is not a polite neighbour, there is no gain at 10 percent random loss, and the tabular agent did not beat AIMD.
- Next: fairness-aware training with TCP coexistence, training on real traces, and controlled multi-machine tests.

43. Thank you

Time on this slide: 0:15



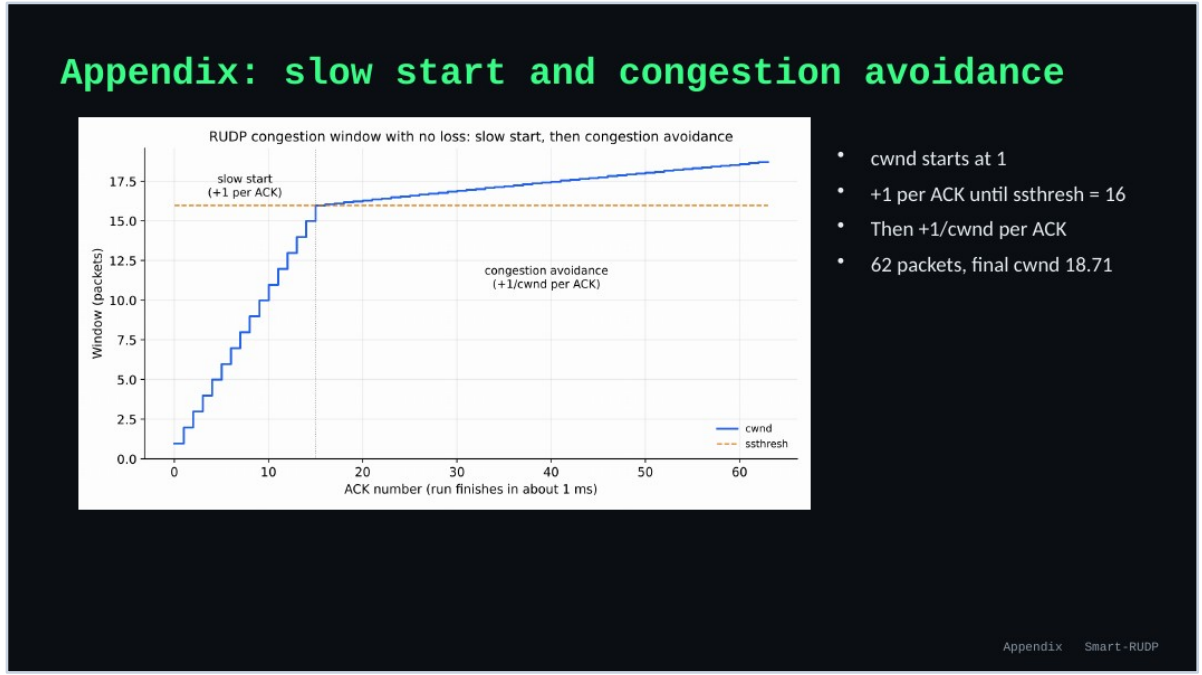
SAY

- Thank you. The code, report and captures are at github.com/Capsturrrr/rudp. Happy to take questions.

IF ASKED Pause and smile; hand over to questions.

44. Appendix: slow start and congestion avoidance

Time on this slide: -



IF ASKED Use this slide to answer 'show me congestion avoidance'.

SAY

- Only if asked: the window starts at 1, adds 1 per ACK until the threshold 16, then 1/window per ACK. In the 62-packet run the final window was 18.71.

Hard questions and short answers (1 of 3)

Is this a lab exercise?

A lab sends datagrams. We built reliability and congestion control on top, measured them, and used them in a real chat and a routed network.

Why not TCP?

TCP is in the kernel and cannot be changed. Building our own makes every mechanism visible. QUIC and HTTP/3 do the same.

Did Smart-RUDP win?

Yes, in the final C transport against tuned baselines: 15 to 52 percent less time than tuned AIMD on four ordinary paths (real clock, real sockets and 100 random held-out paths agree), a tie on the 10 percent loss path. Against a simplified BBR it wins on the satellite-like and fast paths and loses on the 2 percent lossy path. The first tabular agent did not beat AIMD, and we say so.

Why Go-Back-N?

We started with it because it is simple to build and explain, then replaced it as the default with Selective Repeat: the receiver buffers out-of-order packets and reports them in an ACK bitmap, and the sender resends only what is missing. Go-Back-N stays behind a flag for comparison.

Was it tested between machines?

Yes. Delay, loss and jitter are emulated in our client, in a separate proxy on real UDP sockets, and in virtual time; these agree with each other. We also ran it once on the Linux kernel with tc netem on one laptop: the agent beat tuned AIMD there, but kernel TCP was 1.5 to 1.7 times faster than us. We also made one run between two laptops on different networks over a Tailscale relay: RUDP delivered every transfer that finished, but the path drifted and four of twenty runs timed out, so we do not rank the policies from it; kernel TCP was 2 to 6 times faster on the clean repetitions.

Hard questions and short answers (2 of 3)

What does Packet Tracer prove?

The network design: OSPF, VLANs, ACL, 70/71 failover. The protocol is validated separately with emulation and Wireshark.

Peer disappears?

Sender keeps retransmitting on timeout. A connection abort after repeated timeouts is not built; it is the fix we would add.

Does the checksum catch errors?

It is CRC-16/CCITT now. The first version was a byte sum and a fuzzer showed 2 percent of corrupted packets got through; with CRC-16 none of 333,000 did. It cannot catch every error pattern, as no 16-bit check can.

Why no handshake in the exported trace?

The export builds synthetic packets. The real tshark capture shows SYN, SYN-ACK, ACK.

Is the baseline fair?

We tuned it: window cap 128 and a high start threshold, plus the same retransmission-timer floor for every policy. We also compare against tuned CUBIC. Tuned AIMD improved a lot, and the agent is still ahead on four paths.

Hard questions and short answers (3 of 3)

Is it fair to other flows?

No, not entirely: against tuned AIMD it takes 68 percent of a shared link, (CUBIC takes 61 percent). Nothing starves. We did not train for fairness. With unequal round-trip times two neural flows are fairer than two AIMD flows (Jain 0.84 against 0.64 at +150 ms). Against a kernel TCP flow on the same link it takes only 24 to 34 percent, so it does not starve TCP (three runs per cell).

How do you know the simulator is right?

The virtual-time mode runs our real C sender code with a simulated receiver and clock. It matched the real-clock runs in all 15 cells, and the real-socket proxy agrees too.

Is Q-learning really learning?

80 states, 5 actions, 40 episodes; the learning-curve figure shows it. Small and tabular, with limited training.

Did you compare with BBR?

Yes, a simplified BBR-style policy, not the Linux one. It beats tuned AIMD on four of five paths, is faster than our agent on the 2 percent lossy path, and slower on the satellite-like and fast paths. Two bugs in our own version were found and fixed.

Can the neural network be trusted?

In practice it is tolerant: noise of 0.2 on its inputs costs at most 12 percent, and a small decision tree gives the same transfer time on four of five paths. It is not formally verified: certification works only at tiny input changes.